Содержание

Тема 1. «ВЫЧИСЛЕНИЕ ФУНКЦИИ С ПОМОЩЬЮ РАЗЛОЖЕНИЯ В РЯД» . . . 2
1.1 Математическая модель . . . 2
1.2 Проектирование пользовательского интерфейса . . . 3
1.3 Свойства компонентов формы . . . 3
1.4 Сценарий использования программы . . . 4
1.5 Код программы . . . 4
1.6 Схемы основных подпрограмм . . . 7
1.7 Протокол испытаний . . . 10
1.8 Тестирование проекта: . . . 11
1.9 Работа с отладчиком . . . 12
Тема 2. «ИСПОЛЬЗОВАНИЕ ПОДПРОГРАММ ДЛЯ ОБРАБОТКИ ПРОСТЫХ ТИПОВ ДАННЫХ» . . . 13
2.1 Математическая модель . . . 13
2.2 Проектирование пользовательского интерфейса . . . 14
2.3 Свойства компонентов формы . . . 14
2.4 Сценарий использования программы . . . 14
2.5 Код программы . . . 15
2.6 Схемы основных подпрограмм . . . 19
2.7 Протокол испытаний . . . 23
2.8 Тестирование проекта: . . . 23
2.9 Тестирование подпрограмм с помощью тест-кейсов . . . 24
Тема 3. «РАБОТА С ГРАФИКОЙ» . . . 27
3.1 Код программы . . . 28
3.2 Демонстрация . . . 31
Тема 4. «ПОСТРОЕНИЕ ГРАФИКОВ И ДИАГРАММ» . . . 32
4.1 Код программы . . . 33
4.2 Демонстрация . . . 42
Тема 5. «ПРОГРАММЫ ДЛЯ ОБРАБОТКИ ТЕКСТОВЫХ ФАЙЛОВ» . . . 43
5.1 Код программы . . . 43
5.2 Демонстрация . . . 43
Тема 6. «РАЗРАБОТКА ИГРОВОГО ПРИЛОЖЕНИЯ НА ОСНОВЕ XML-ФАЙЛА» . . . 44
6.1 Сценарий использования программы . . . 45
6.2 Код программы . . . 45
6.3 Демонстрация . . . 49
Тема 7. «ВЕКТОРНЫЙ РЕДАКТОЕ» . . . 51
7.1 Математическая модель . . . 51
7.2 Проектирование пользовательского интерфейса . . . 52
7.3 Свойства компонентов формы . . . 52
7.4 Сценарий использования программы . . . 52
7.5 Код программы . . . 52
7.6 Схемы основных подпрограмм . . . 52
7.7 Протокол испытаний . . . 52
7.8 Тестирование проекта: . . . 52
7.9 Тестирование проекта: . . . 52
Тема 8. «РАЗРАБОТКА ИГРЫ С ЭЛЕМЕНТАМИ ГРАФИКИ» . . . 52
8.1 Код программы . . . 53
8.2 Демонстрация . . . 60

Тема 1. «ВЫЧИСЛЕНИЕ ФУНКЦИИ С ПОМОЩЬЮ РАЗЛОЖЕНИЯ В РЯД»

Цель работы: получить практические навыки разработки проектов для вычисления значения функции при помощи разложения в ряд (рядов Тейлора). Отработать навыки работы с отладчиком.

Задание 1. Необходимо разработать программу, вычисляющую значение функции с помощью разложения в ряд. Сумма ряда вычисляется при помощи цикла с неизвестным числом повторений, так как требуется найти значение с заданной точностью (допустимая погрешность вводится с клавиатуры). Сходящийся степенной ряд будет достигать искомого значения при достаточно большом количестве суммируемых членов ряда. При этом вклад последнего слагаемого в сумму будет пренебрежимо малым.Для проверки полученного результата выведите значение указанной функции (левую часть выражения), вычисленную в цикле сумму ряда (правую часть выражения), а также количество просуммированных членов ряда.

Задание 2. Необходимо добавьть

Проверку на:

- правильности ввода данных при нажатии клавиш клавиатуры в полях редактирования (используйте событие KeyPress);
- диапазона вводимых данных;
- контроль пустых полей ввода (заблокируйте кнопку «Вычислить», пока не будут введены все данные).

Вариант 5

$$\arcsin(x) - x = \sum_{n=1}^{\infty} \frac{1 \cdot 3 \cdot 5 \cdot \dots \cdot (2n-1) \cdot x^{2n+1}}{2 \cdot 4 \cdot 6 \cdot \dots \cdot 2n \cdot (2n+1)} =$$

$$= \frac{x^3}{2 \cdot 3} + \frac{1 \cdot 3 \cdot x^5}{2 \cdot 4 \cdot 5} + \frac{1 \cdot 3 \cdot 5 \cdot x^7}{2 \cdot 4 \cdot 6 \cdot 7} + \dots, \quad |x| < 1$$

Выполнение заданий

1.1. Математическая модель

Дано:

$x, eps \in \mathbb{R}$ (x — аргумент, eps — допустимая погрешность).

Найти:

$\text{arcResult}, \text{sum} \in \mathbb{R}$

$n \in \mathbb{N}$

Связь:

$(1) \to \text{arcResult} = \arcsin(x) - x$

(2) $\rightarrow \text{sum} = S_{n-1} = \sum_{k=0}^{n-1} a_k$, где

(3) $\rightarrow a_k = \frac{1 \cdot 3 \cdot 5 \cdot \ldots \cdot (2k-1)}{2 \cdot 4 \cdot 6 \cdot \ldots \cdot 2k \cdot (2k+1)} \cdot x^{2k+1}, k = 0, \ldots, n$

(4) $\rightarrow n : |a_{n-1}| \leq eps \cdot |S_{n-1}|$

Вместо формул (2) и (3) используем рекуррентные формулы (2*)–(5*)

(2*) $\rightarrow S_k = \begin{cases} 0, k=0 \\ S_{k-1} + a_k, k=1,2,\ldots,n \end{cases}$

(3*) $\rightarrow a_k = \frac{b_k}{2k+1}$, где

(5) $\rightarrow b_k = \frac{2k-1}{2k} \cdot x^2 \cdot b_{k-1}$

(5*) $\rightarrow b_k = \begin{cases} x, k=0 \\ \frac{2k-1}{2k} \cdot x^2 \cdot b_{k-1}, k=1,2,\ldots,n \end{cases}$ v(0em)

ОДЗ:

$-1 \leq x \leq 1, 0 < eps < 1$

1.2. Проектирование пользовательского интерфейса

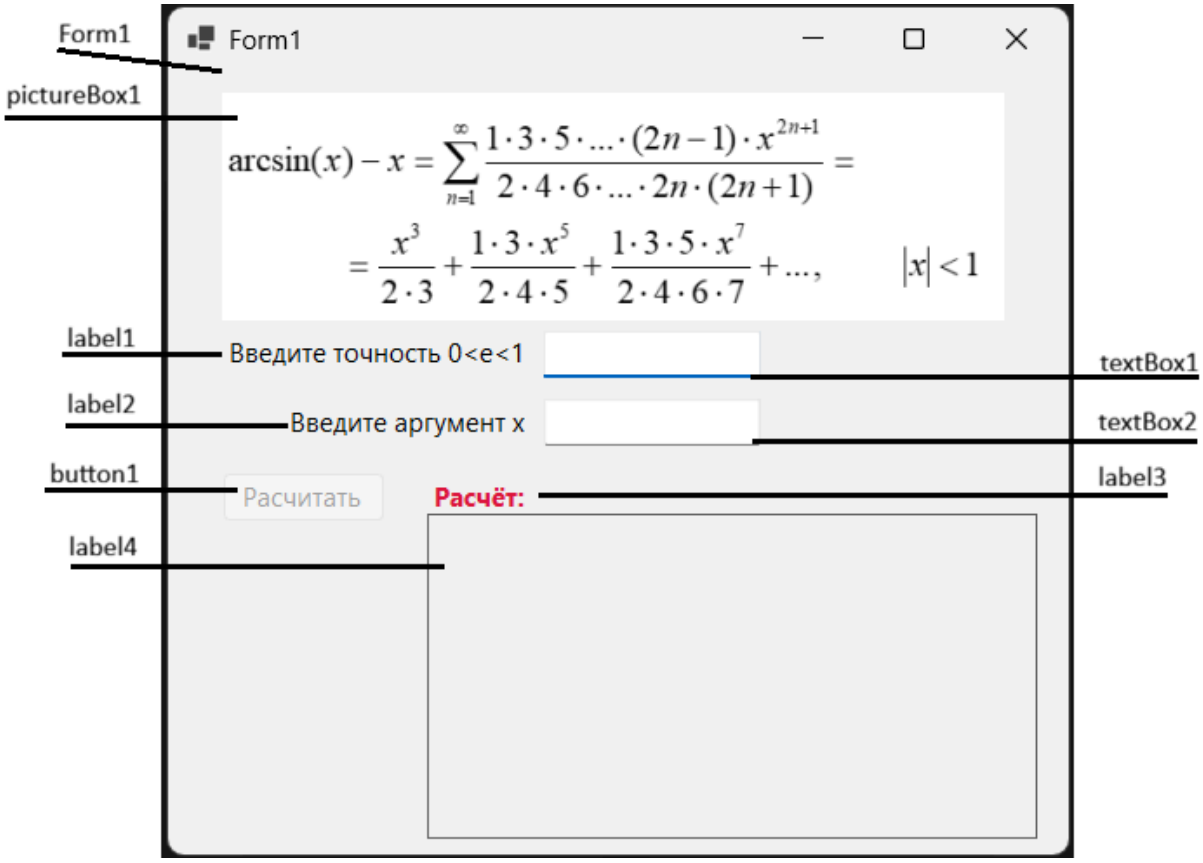


Рисунок 1 - Блок-схема алгоритма

1.3. Свойства компонентов формы

Таблица 1 - Значения свойств, установленные на этапе конструирования интерфейса

Название компонента	Свойства	Значения
Form1	Text	Form1
	Font	Segoe UI, 9pt
pictureBox1	SizeMode	Zoom
	Image	task1.png
label1	Text	Введите точность 0<e<1
label2		Введите аргумент x
label3		Расчёт:
label4	BorderStyle	FixedSingle
button1	Text	Расчитать

1.4. Сценарий использования программы

1. Ввод x и eps в полях ввода.
2. Нажатие на кнопку Вычислить.
3. Вычисление значений arcResult, sum и n - количество элементов, необходимых для вычисления значения sum.
4. Возможность вернуться к пунктам 1 и/или 2.
5. Завершение работы программы.

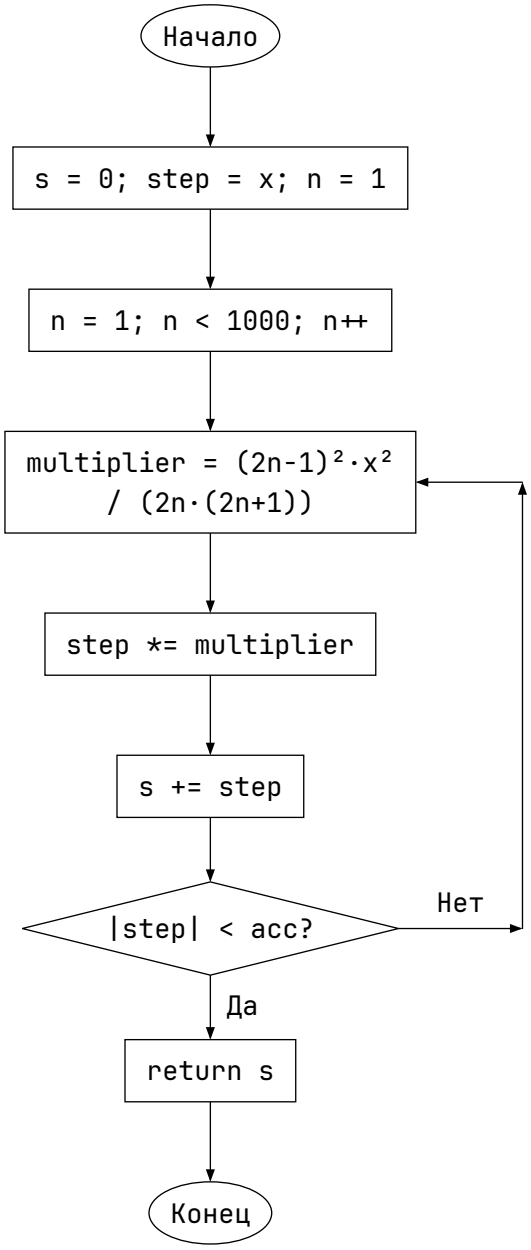
1.5. Код программы

```
namespace UP
{
    public partial class Form1 : Form
    {
        private double arcsin_iterator(out int n, double x, double acc)
        {

            double s = 0;
            double step = x;
            n = 1;

            do
            {
                double multiplier = (Math.Pow(2 * n - 1, 2) * x * x) /
((2 * n) * (2 * n + 1));
                step *= multiplier;

                s += step;
```

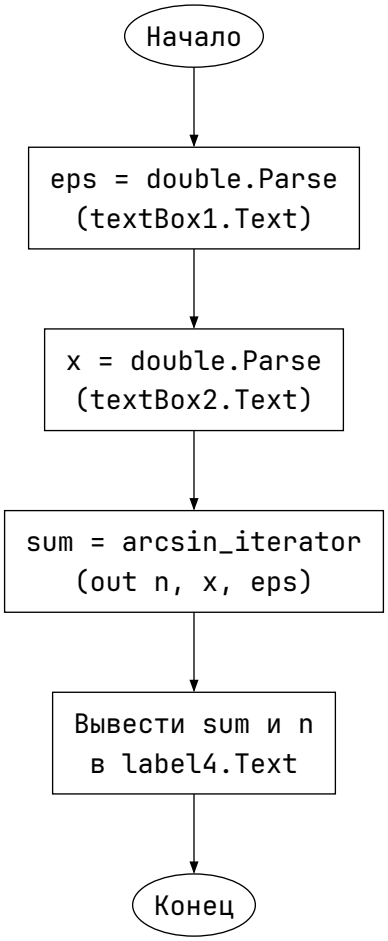
```
                n++;

            } while (Math.Abs(step) ≥ acc && n < 1000);
            return s;

        }
        public Form1()
        {
            InitializeComponent();
        }

        private void textBox1_TextChanged(object sender, EventArgs e)
        {
            try
            {
                double.Parse(textBox2.Text);
                if (double.Parse(textBox1.Text) ≥ 1e-16 &&
double.Parse(textBox1.Text) < 1)
                    button1.Enabled = true;

                else button1.Enabled = false;
            }
            catch
            {
                button1.Enabled = false;
            }
        }

        private void textBox1_KeyPress(object sender, KeyPressEventArgs
e)
        {

            if (e.KeyChar ≥ '0' && e.KeyChar ≤ '9')
                return;

            if (e.KeyChar == (char)Keys.Back && textBox1.Text ≠ "")
                return;

            if (e.KeyChar == ',' || e.KeyChar == '.')
            {
                e.KeyChar = '.';
                try
                {
                    double.Parse($"{textBox1.Text}{e.KeyChar}");
                    return;
```

```
                }
                catch { }
            }

            e.KeyChar = '\0';
        }

        private void textBox2_KeyPress(object sender, KeyPressEventArgs
e)
        {

            if (e.KeyChar ≥ '0' && e.KeyChar ≤ '9')
                return;

            if (e.KeyChar == (char)Keys.Back && textBox2.Text ≠ "")
                return;

            if (e.KeyChar == '-' && textBox2.Text == "")
                return;

            if (e.KeyChar == ',' || e.KeyChar == '.')
            {
                e.KeyChar = '.';
                try
                {
                    double.Parse($"{textBox2.Text}{e.KeyChar}");
                    return;
                }
                catch { }
            }

            e.KeyChar = '\0';
        }

        private void button1_Click(object sender, EventArgs e)
        {
            double sum = arcsin_iterator(out int series,
double.Parse(textBox2.Text), double.Parse(textBox1.Text));
            label4.Text = $"Arcsin(x) - x = {sum.ToString()}\n" +
                $"Сумма ряда: {sum.ToString()}\n" +
                $"Количество членов ряда: {(int)
(decimal.GetBits(decimal.Parse(textBox1.Text))[3] >> 16) & 31}";
        }
    }
}
```

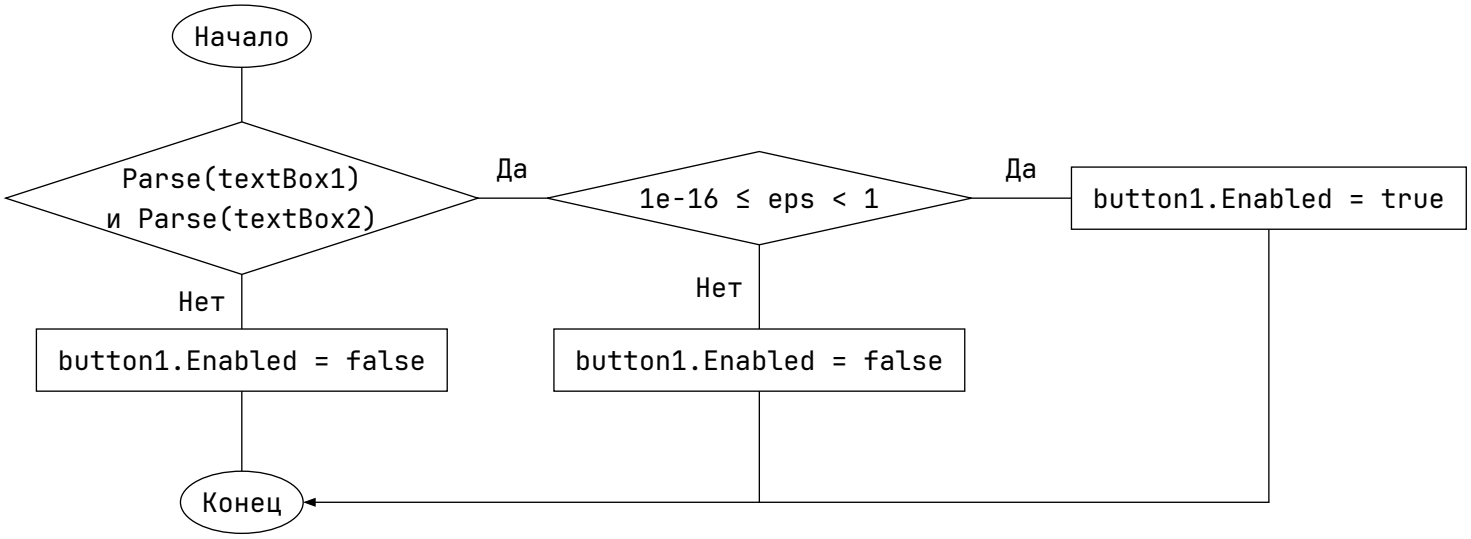
1.6. Схемы основных подпрограмм



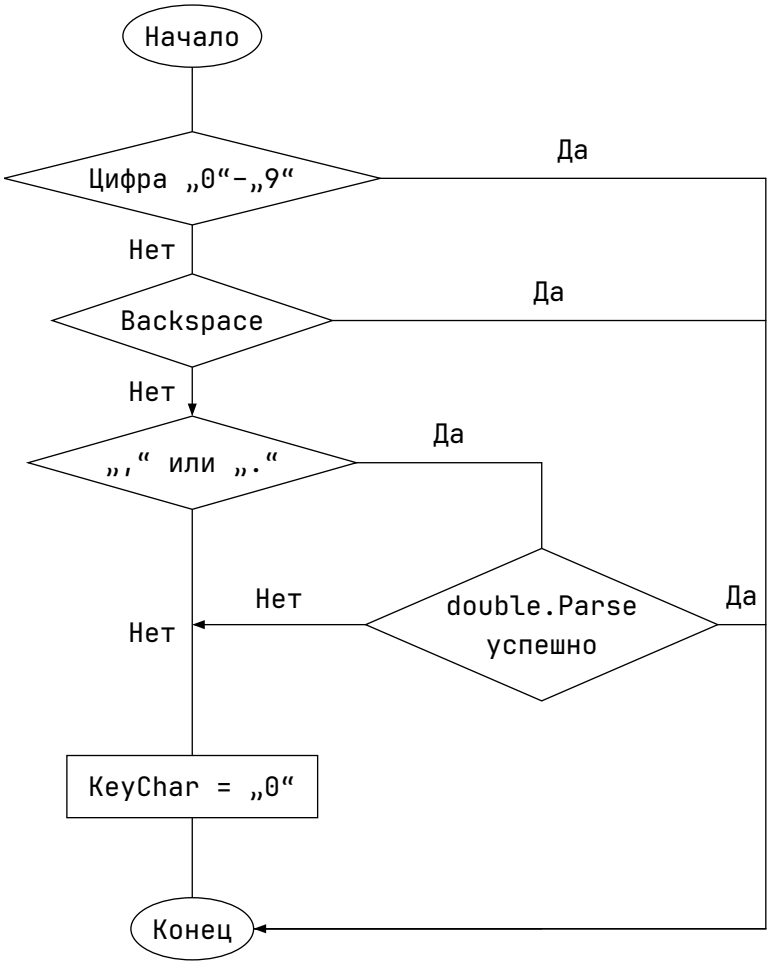
Блок-схема 1 — arcsin_iterator



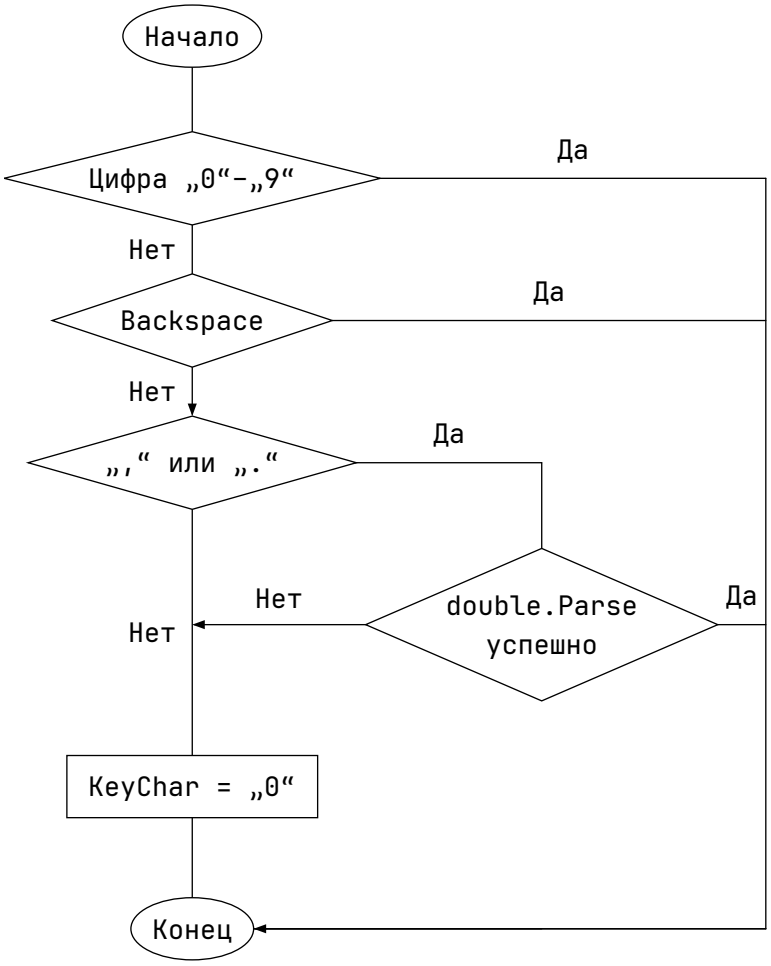
Блок-схема 2 — button1_Click



Блок-схема 3 — textBox1_TextChanged



Блок-схема 4 — textBox1_KeyPress



Блок-схема 5 — textBox1_KeyPress

1.7. Протокол испытаний

Таблица 2 - Протокол спытаний

Проверяемые требования	**Сообщения программы и вводимые значения**	**Ожидаемые результаты**	**Фактические результаты**
Способность ввода данных	e = 0.005; x = −0.2	Числа вводятся в текстовое поле	Такие же, как ожидаемые результаты **(См. Рисунок 2)**
Способнось контроля вводимых данных	e = D0c; x = mJ7	из-за 0 точности кнопка не загорится e = 0; x = 7	Такие же, как ожидаемые результаты **(См. Рисунок 4)**
Способность программы обеспечить контроль вводимых данных	e = −0,,0-05-; x = −0..994-	из-за 0 точности кнопка не загорится e = 0.005; x = −0.994	Такие же, как ожидаемые результаты **(См. Рисунок 3)**

1.8. Тестирование проекта:

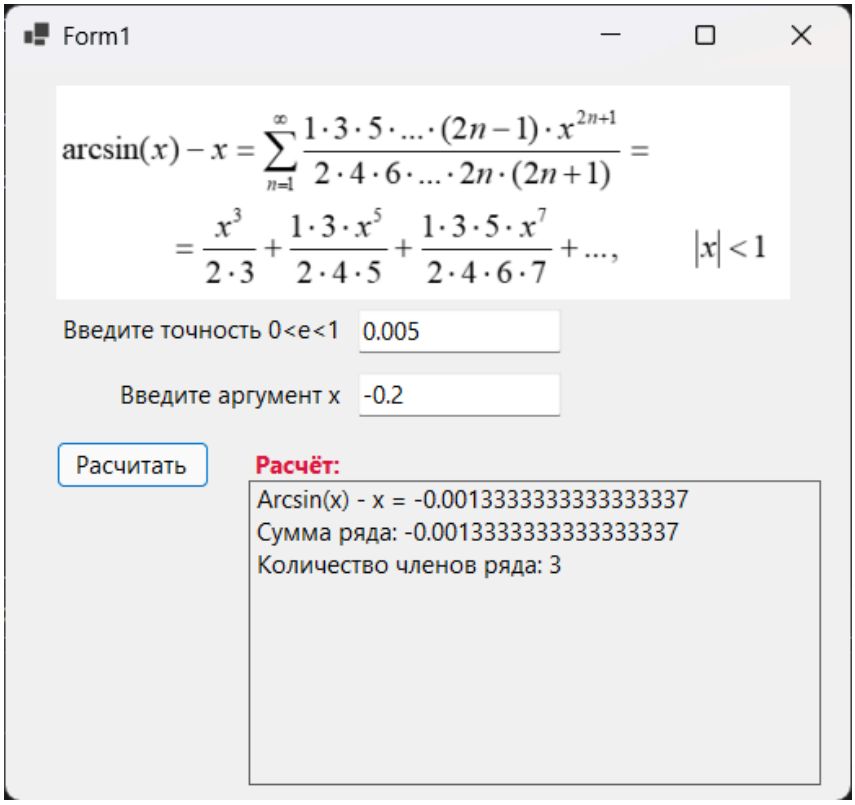


Рисунок 2 - Демонстрация теста

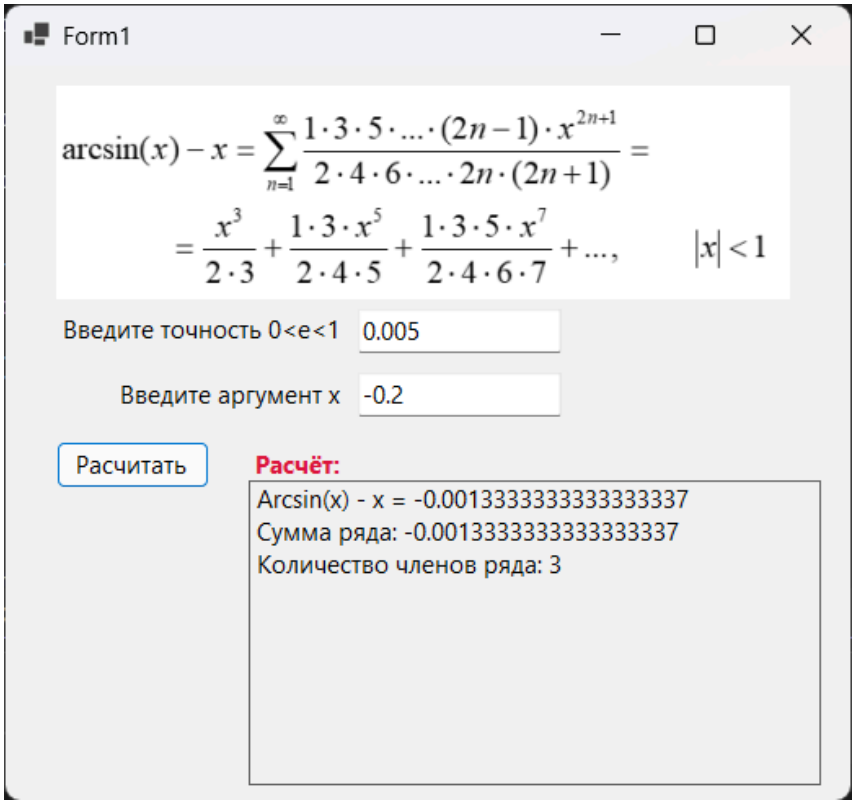


Рисунок 3 - Демонстрация теста

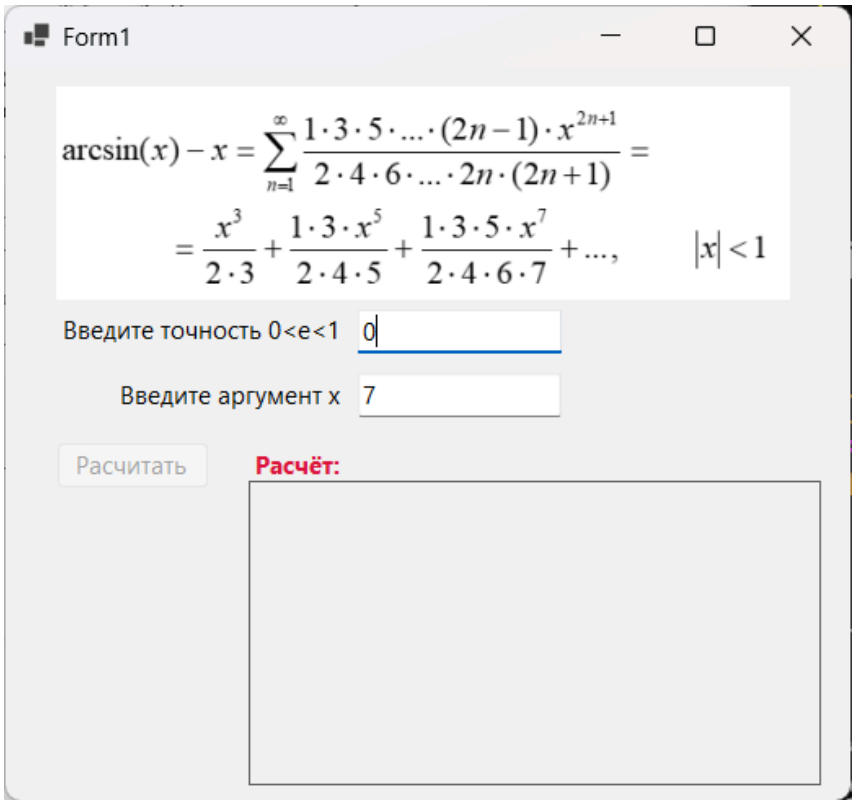


Рисунок 4 - Демонстрация теста

1.9. Работа с отладчиком

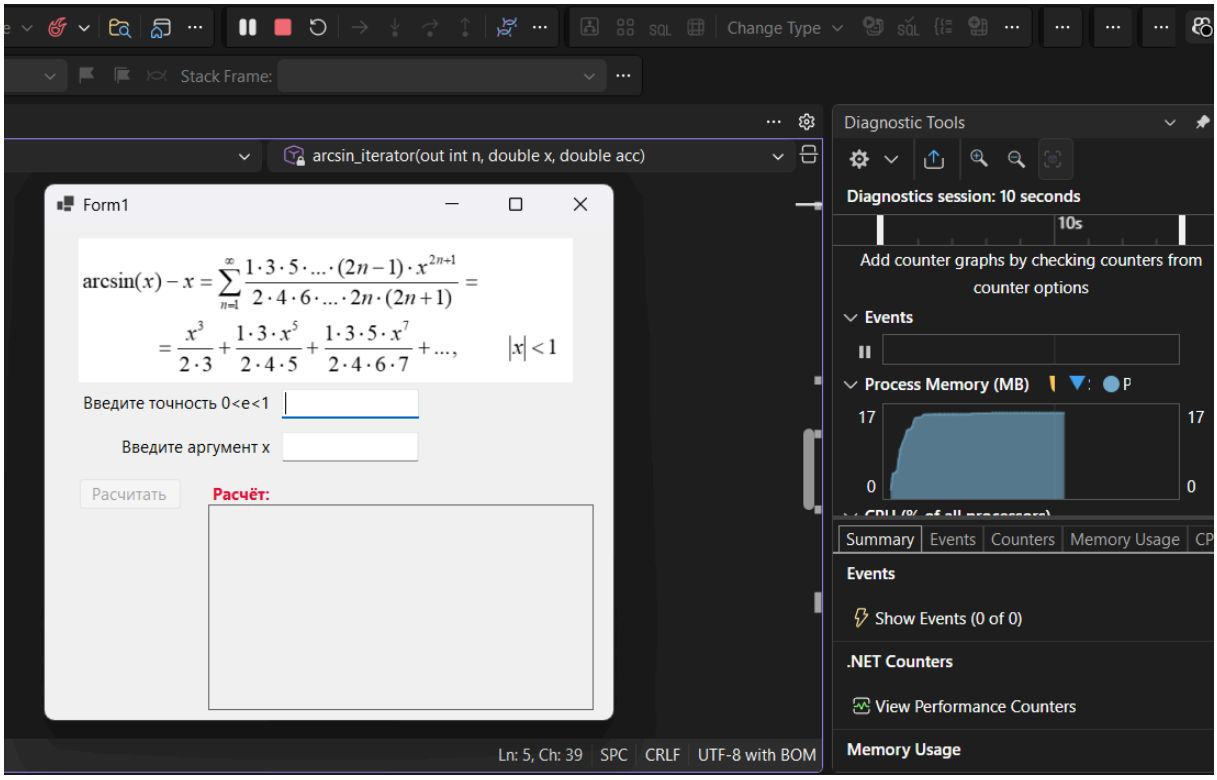


Рисунок 5 - Работа с отладчиком

Тема 2. «ИСПОЛЬЗОВАНИЕ ПОДПРОГРАММ ДЛЯ ОБРАБОТКИ ПРОСТЫХ ТИПОВ ДАННЫХ»

Цель работы: получить практические навыки разработки спецификаций подпрограмм для обработки простых типов данных, кодирования и вызова собственных подпрограмм, тестирования созданных подпрограмм по потоку данных.

Задание 1. Создание дополнительного класса и разработка подпрограмм (методов) класса.

Задание 2. Тестирование подпрограмм с помощью юнит-тестов

- Подготовить тестовый сценарий и тестовый набор данных методом «чёрного ящика» (тестирование по потоку данных);
- Для тестирования методов класса создать проект UnitTestsProject.

При создании тестового проекта добавляйте его в существующее решение;

- В тестовом проекте создать функции тестирования, в том числе при допустимых и недопустимых исходных данных (негативный тест), рассмотреть стандартные и крайние случаи;
- Запустить юнит-тесты и продемонстрировать успешное тестирование.

Задание 3. Создание пользовательского интерфейса (модуля формы) Необходимо настроить внешний вид и поведение компонентов формы и создать методы обработки событий. Модуль формы должен содержать код, который продемонстрирует вызов методов из дополнительного класса по вашему заданию. Оконный интерфейс может быть самым простым, важно чтобы можно было удобным образом задать исходные данные и проверить вызов ваших методов. В случае, если в задании требуется получать исходные данные из файла или же сохранять результаты в файл, добавьте в проект диалоговое окно открытия или сохранения файла.

Вариант 5

1. Разработать подпрограмму для определения, в каких четвертях координатной плоскости лежит отрезок. Отрезок на плоскости задается координатами своих концов.
2. Построить координатную плоскость и графический объект (отрезок с заданными параметрами). Рисунок будет служить для контроля правильности решения.
3. Используя созданные подпрограммы, определить в каких координатных четвертях находится отрезок, заданный пользователем, и выдать соответствующее сообщение.

Выполнение заданий

2.1. Математическая модель

Дано:

две точки описывающие отрезок - a и b своими координатами (a.x,a.y) и (b.x,b.y).

x и y - вещественные числа

бинарное переменная c для хранения маски четвертей

Найти:

четверти системы координат в которых лежит отрезок

Связь:

параметрическое уравнение отрезка

ОДЗ:

дополнительных ограничений нет

2.2. Проектирование пользовательского интерфейса

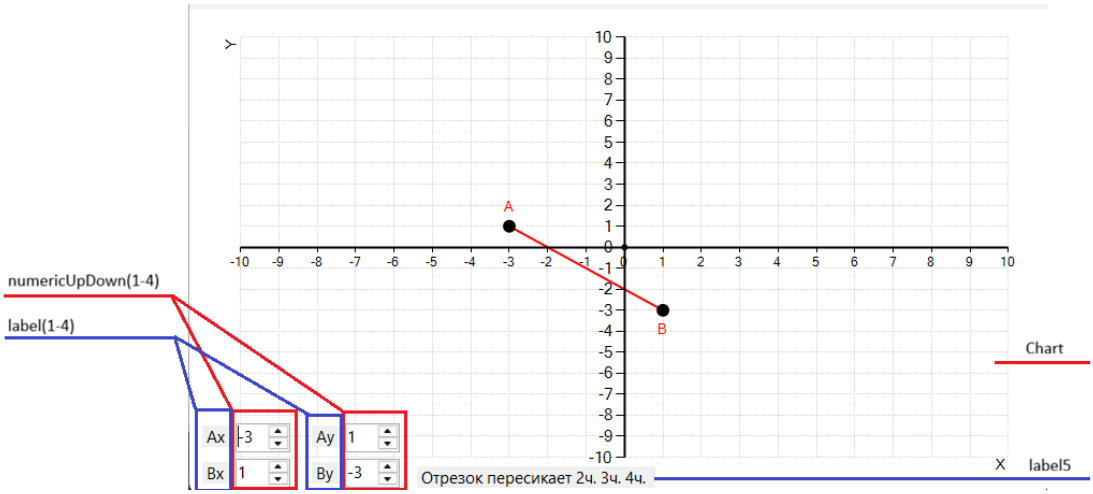


Рисунок 6 - Форма

2.3. Свойства компонентов формы

Таблица 3 - Значения свойств, установленные на этапе конструирования интерфейса

Название компонента	Свойства	Значения
Form1	Text	Form1
	Font	Segoe UI, 9pt
Chart	Dock	Fill
label1	Text	Ax
label2		Bx
label3		By
label4		Ay
label5		Отрезок пересикает

2.4. Сценарий использования программы

1. Открывается форма
2. Загружается график
3. Изменяются положения точек отрезка
4. Выводятся четверти в которых отрезок лежитё

2.5. Код программы

Класс работы над отрезком

```
public class Segment
{
    public record struct Point(float X, float Y);

    public Point A, B;
    private ISegment _form;

    public Segment(ISegment form, float x1, float y1, float x2, float
y2)
    {
        _form = form;
        UpdatePoint(x1, y1, x2, y2);
    }
    public void UpdatePoint(float x1, float y1, float x2, float y2)
    {
        A.X = x1;
        A.Y = y1;
        B.X = x2;
        B.Y = y2;
    }
    public void GetQuadrant()
    {
        float dX = B.X - A.X, dY = B.Y - A.Y;
        float f = A.X + (-A.Y / (B.Y - A.Y)) * (B.X - A.X);
        byte c = 0;

        if (B.X != 0 && B.Y != 0)
            c |= (B.X > 0, B.Y > 0) switch
            {
                (true, true) => 2,
                (false, true) => 4,
                (false, false) => 8,
                (true, false) => 16,
            };

        if (A.X != 0 && A.Y != 0)
            c |= (A.X > 0, A.Y > 0) switch
            {
```

```
                (true, true) => 2,
                (false, true) => 4,
                (false, false) => 8,
                (true, false) => 16,
            };

        if (dY != 0)
        {
            float t = -A.Y / dY;
            float crossX = A.X + t * dX;
            if (t > 0 && t < 1)
            {
                if (crossX > 0) c |= 18;
                if (crossX < 0) c |= 12;
            }
        }

        if (dX != 0)
        {
            float t = -A.X / dX;
            float crossY = A.Y + t * dY;
            if (t > 0 && t < 1)
            {
                if (crossY > 0) c |= 6;
                if (crossY < 0) c |= 24;
            }
        }
        string mass = "Отрезок пересикает ";
        if ((c & 2) != 0) mass += "1ч. ";
        if ((c & 4) != 0) mass += "2ч. ";
        if ((c & 8) != 0) mass += "3ч. ";
        if ((c & 16) != 0) mass += "4ч. ";
        _form.PrintQuadrant(mass);
    }


}
```

Интерфейс связывающий форму с отрезком

```
public interface ISegment
{
    void PrintQuadrant(string text);
}
```

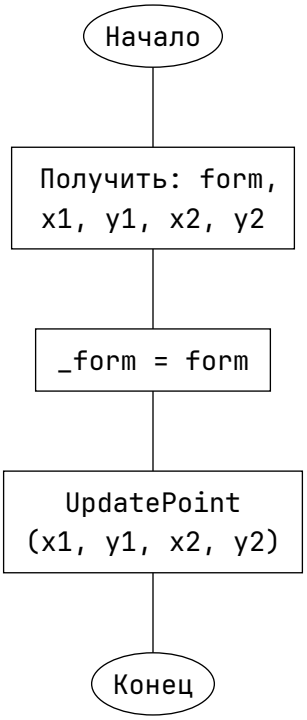
Класс формы

```
public partial class Form1 : Form, ISegment
{
    private Chart _chart;

    private Segment _segment;
    public Form1()
    {
        InitializeComponent();
        _segment = new Segment(this,
            (float)numericUpDown1.Value,
            (float)numericUpDown4.Value,
            (float)numericUpDown2.Value,
            (float)numericUpDown3.Value
            );
        InitChart();
        DrawSegment();
    }

    public void PrintQuadrant(string text)
    {
        label5.Text = text;
    }
    private void InitChart()
    {

        _chart = new Chart { Dock = DockStyle.Fill };

        var area = new ChartArea("Main");


        // Ось X
        area.AxisX.Minimum = -10;
        area.AxisX.Maximum = 10;
        area.AxisX.Interval = 1;
        area.AxisX.Crossing = 0;
        area.AxisX.Title = "X";
        area.AxisX.TitleAlignment = StringAlignment.Far;
        area.AxisX.LineColor = Color.Black;
        area.AxisX.LineWidth = 2;
        area.AxisX.MajorTickMark.Enabled = true;
        area.AxisX.MajorGrid.LineColor = Color.LightGray;
        area.AxisX.MajorGrid.LineDashStyle = ChartDashStyle.Dot;

        // Ось Y
        area.AxisY.Minimum = -10;
        area.AxisY.Maximum = 10;
```

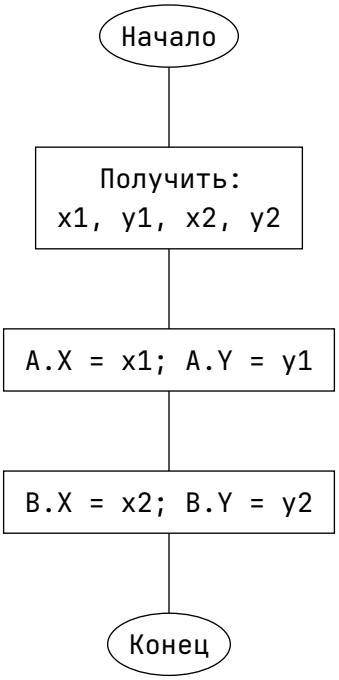
```
        area.AxisY.Interval = 1;
        area.AxisY.Crossing = 0;
        area.AxisY.Title = "Y";
        area.AxisY.TitleAlignment = StringAlignment.Far;
        area.AxisY.LineColor = Color.Black;
        area.AxisY.LineWidth = 2;
        area.AxisY.MajorTickMark.Enabled = true;
        area.AxisY.MajorGrid.LineColor = Color.LightGray;
        area.AxisY.MajorGrid.LineDashStyle = ChartDashStyle.Dot;

        area.BackColor = Color.White;

        _chart.ChartAreas.Add(area);
        Controls.Add(_chart);

    }

    private void DrawSegment()
    {

        _segment.UpdatePoint((float)numericUpDown1.Value,
            (float)numericUpDown4.Value,
            (float)numericUpDown2.Value,
            (float)numericUpDown3.Value
            );

        _chart.Series.Clear();

        var segment = new Series()
        {
            ChartType = SeriesChartType.Line,
            Color = Color.Red,
            BorderWidth = 2,
            ChartArea = "Main",

            MarkerStyle = MarkerStyle.Circle,
            MarkerSize = 10,
            MarkerColor = Color.Black,

        };

        segment.Points.AddXY(_segment.A.X, _segment.A.Y);
        segment.Points.AddXY(_segment.B.X, _segment.B.Y);

        segment.Points[0].Label = $"A";
        segment.Points[1].Label = $"B";
```

```
        segment.Points[1].ToolTip = "X = #VALX, Y = #VALY";
        segment.Points[0].ToolTip = "X = #VALX, Y = #VALY";
        segment.Points[0].LabelForeColor = Color.Red;
        segment.Points[1].LabelForeColor = Color.Red;

        _chart.Series.Add(segment);


        var origin = new Series()
        {
            ChartType = SeriesChartType.Point,
            MarkerStyle = MarkerStyle.Circle,
            MarkerSize = 5,
            MarkerColor = Color.Black,
            IsVisibleInLegend = false
        };
        origin.Points.AddXY(0, 0);
        _chart.Series.Add(origin);

        _segment.GetQuadrant();
    }

    private void numericUpDown1_ValueChanged(object sender, EventArgs
e)
    {
        if (numericUpDown1.Value > 10) numericUpDown1.Value = 10;
        if (numericUpDown2.Value > 10) numericUpDown2.Value = 10;
        if (numericUpDown3.Value > 10) numericUpDown3.Value = 10;
        if (numericUpDown4.Value > 10) numericUpDown4.Value = 10;

        if (numericUpDown1.Value < -10) numericUpDown1.Value = -10;
        if (numericUpDown2.Value < -10) numericUpDown2.Value = -10;
        if (numericUpDown3.Value < -10) numericUpDown3.Value = -10;
        if (numericUpDown4.Value < -10) numericUpDown4.Value = -10;

        DrawSegment();

        if ((c & 8) != 0) label5.Text += "3ч. ";
    }
}
```

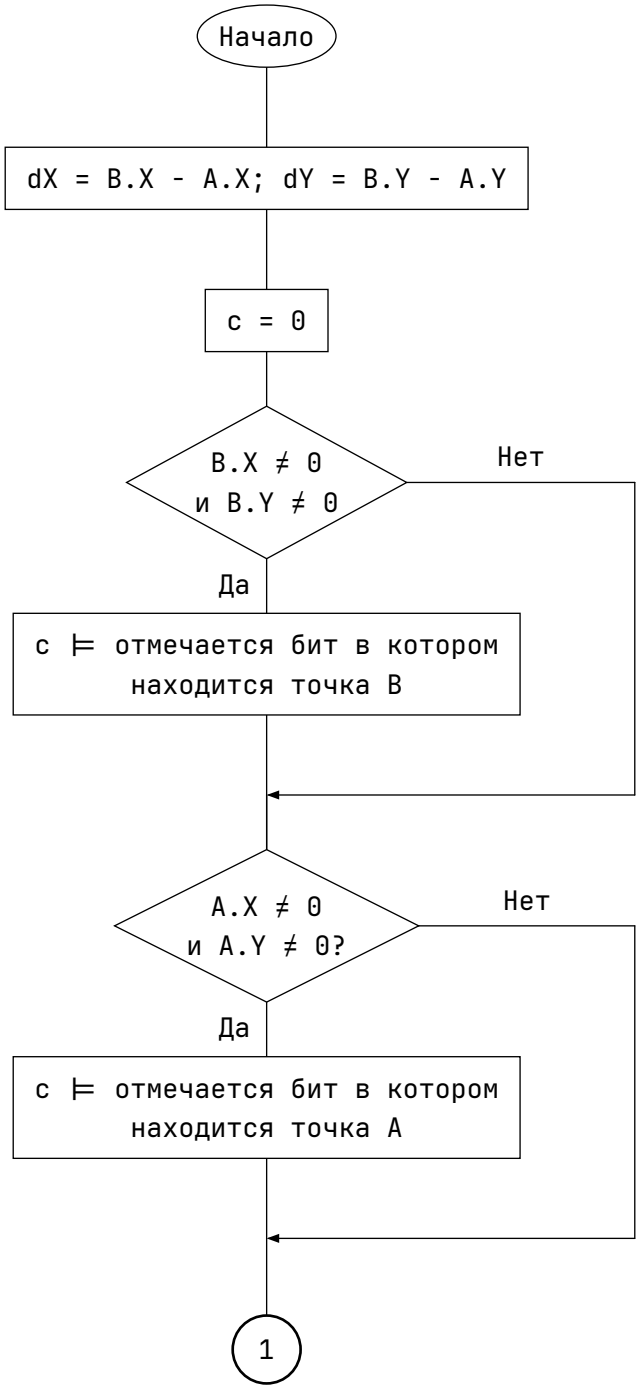
2.6. Схемы основных подпрограмм



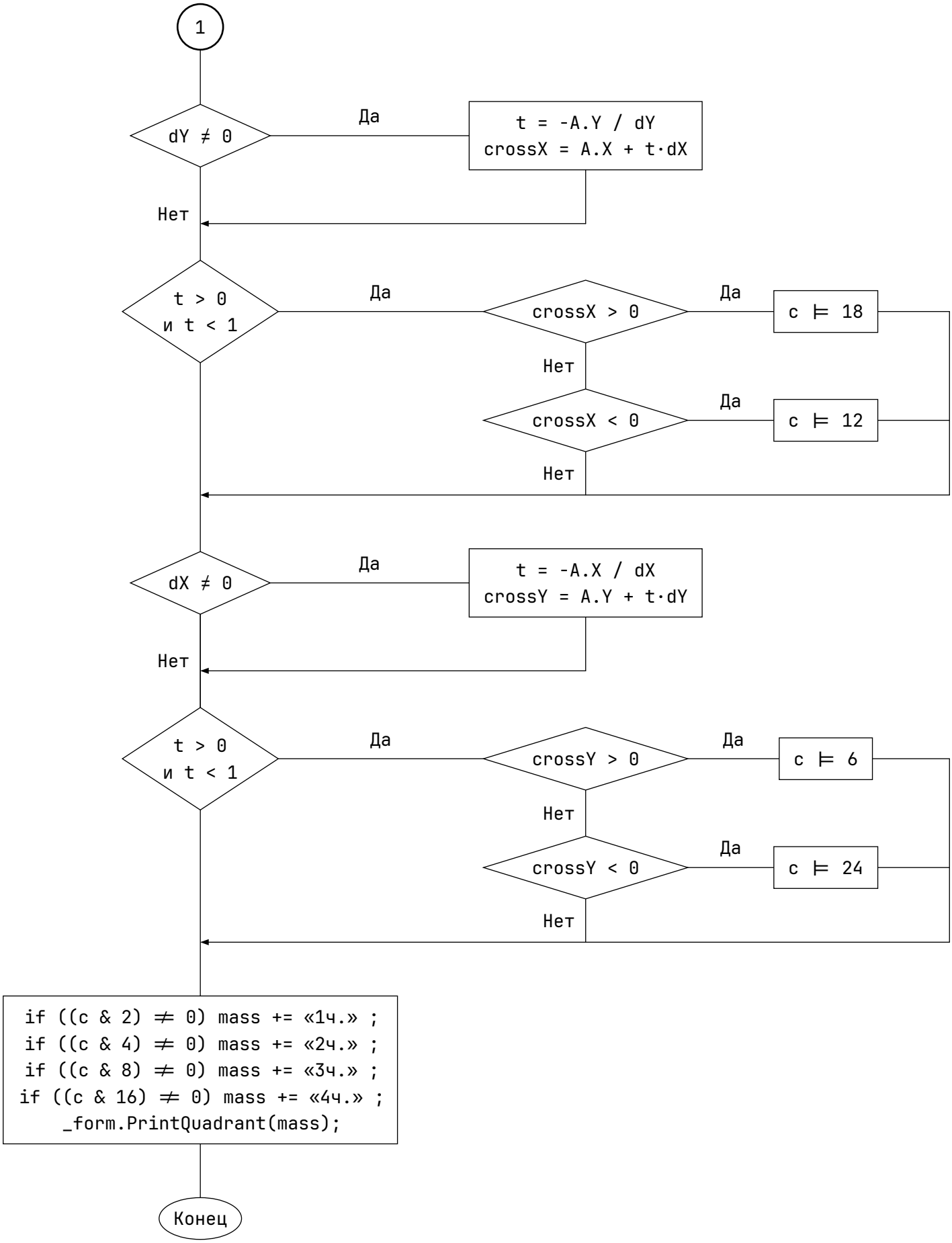
Блок-схема 6 — Конструктор Segment



Блок-схема 7 — UpdatePoint



Блок-схема 8 — GetQuadrant



Блок-схема 9 — GetQuadrant

2.7. Протокол испытаний

Таблица 4 - Протокол спытаний

Проверяемые требования	**Сообщения программы и вводимые значения**	**Ожидаемые результаты**	**Фактические результаты**
Способность ввода данных	Ax = −3 Ay = 1 Bx = 1 By = −3	Числа вводятся в текстовое поле	Такие же, как ожидаемые результаты **(См. Рисунок 6)**
Способность программы обеспечить контроль вводимых данных	Ax = d-G3g Ay = d1 Bx = gej1 By = −3gd	Ax = −3 Ay = 1 Bx = 1 By = −3	Такие же, как ожидаемые результаты **(См. Рисунок 7)**
Способнось контроля вводимых данных	Ax = −58349 Ay = 378 Bx = 2 By = −85	Ax = −10 Ay = 10 Bx = 2 By = −10	Такие же, как ожидаемые результаты **(См. Рисунок 8)**

2.8. Тестирование проекта:

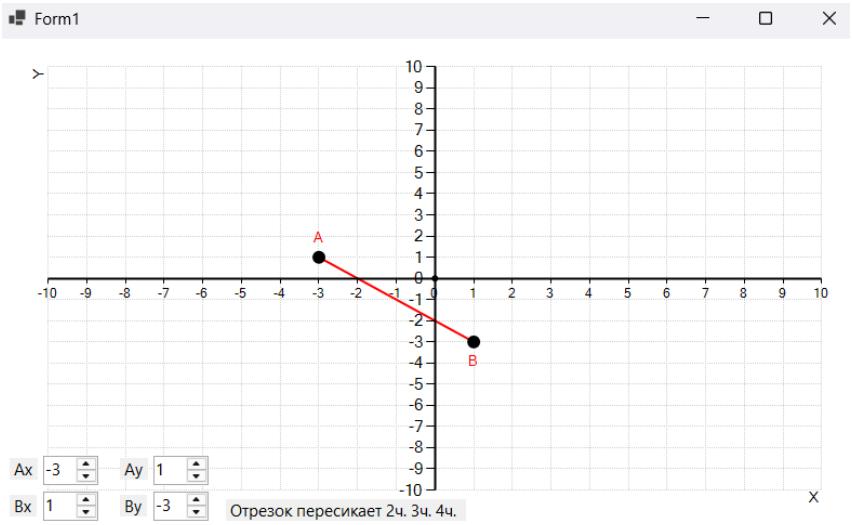


Рисунок 7 - Перемещение и изменение отрезка

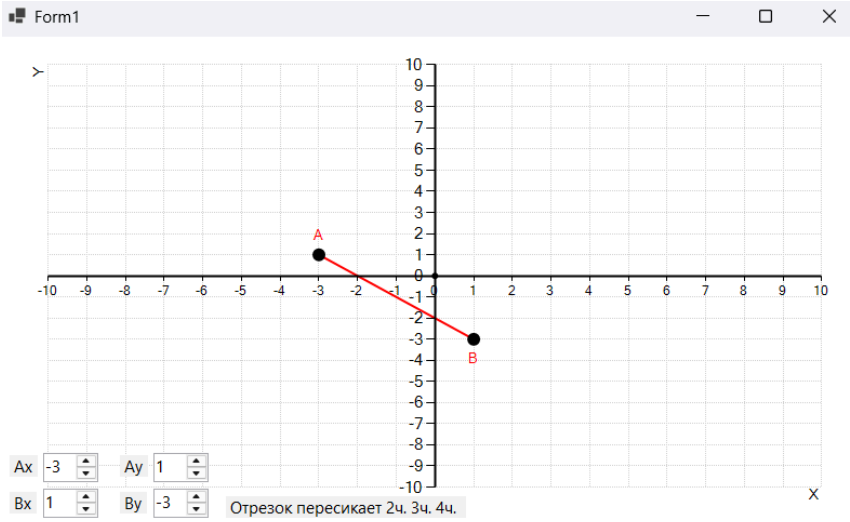


Рисунок 8 - Перемещение и изменение отрезка

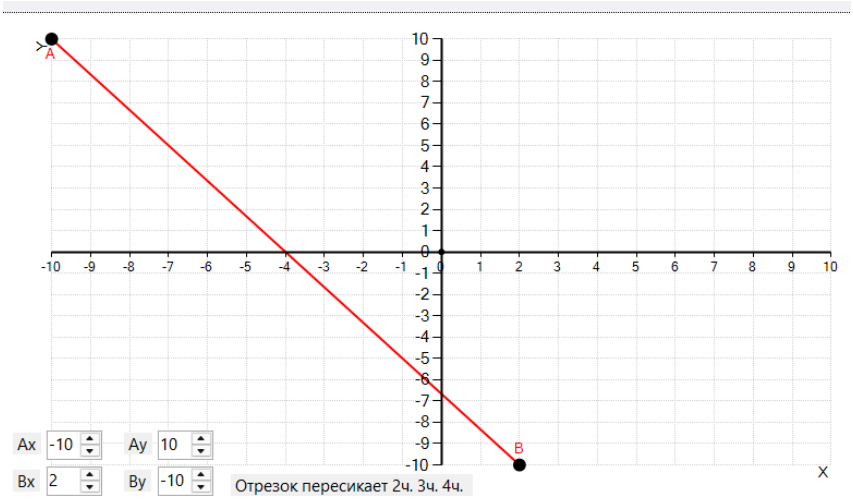


Рисунок 9 - Перемещение и изменение отрезка

2.9. Тестирование подпрограмм с помощью тест-кейсов

```
public class MockSegmentForm : ISegment
{
    public string LastPrinted { get; private set; } = "";
    public void PrintQuadrant(string text) => LastPrinted = text;
}

public class SegmentTests
{
    private readonly MockSegmentForm _mock;

    public SegmentTests()
    {
        _mock = new MockSegmentForm();
    }
```

```
    [Theory]
    [InlineData(2, 3, 4, 1, "1ч.")]
    [InlineData(-2, 3, -4, 1, "2ч.")]
    [InlineData(-2, -3, -4, -1, "3ч.")]
    [InlineData(2, -3, 4, -1, "4ч.")]
    public void
GetQuadrant_BothInSameQuadrant_ReturnsOnlyThatQuadrant(
        float ax, float ay, float bx, float by, string expected)
    {
        var seg = new Segment(_mock, ax, ay, bx, by);
        var result = seg.GetQuadrant();

        Assert.Contains(expected, result);
        foreach (var q in new[] { "1ч.", "2ч.", "3ч.", "4ч." })
            if (q != expected)
                Assert.DoesNotContain(q, result);
    }

    [Theory]
    [InlineData(-2, 2, 2, 2, "1ч.", "2ч.")]
    [InlineData(-2, -2, 2, -2, "3ч.", "4ч.")]
    [InlineData(2, 2, 2, -2, "1ч.", "4ч.")]
    [InlineData(-2, 2, -2, -2, "2ч.", "3ч.")]
    public void GetQuadrant_CrossesOneAxis_ReturnsTwoQuadrants(
        float ax, float ay, float bx, float by,
        string q1, string q2)
    {
        var seg = new Segment(_mock, ax, ay, bx, by);
        var result = seg.GetQuadrant();

        Assert.Contains(q1, result);
        Assert.Contains(q2, result);
    }

    [Fact]
    public void GetQuadrant_CrossesBothAxes_ReturnsThreeQuadrants()
    {
        var seg = new Segment(_mock, -3, 2, 4, -1);
        var result = seg.GetQuadrant();

        Assert.Contains("1ч.", result);
        Assert.Contains("2ч.", result);
        Assert.Contains("4ч.", result);
        Assert.DoesNotContain("3ч.", result);
    }
```

```
    [Fact]
    public void GetQuadrant_ThroughOrigin_ReturnsAllQuadrants()
    {
        var seg = new Segment(_mock, -3, 3, 3, -3);
        var result = seg.GetQuadrant();

        Assert.DoesNotContain("1ч.", result);
        Assert.Contains("2ч.", result);
        Assert.DoesNotContain("3ч.", result);
        Assert.Contains("4ч.", result);
    }

    [Theory]
    [InlineData(3, 0, 3, 3, "1ч.", "4ч.")]
    [InlineData(0, 3, 3, 3, "1ч.", "2ч.")]
    [InlineData(-3, 0, -3, 3, "2ч.", "3ч.")]
    public void GetQuadrant_PointOnAxis_NotCountedAsQuadrant(
    float ax, float ay, float bx, float by,
    string shouldContain, string shouldNotContain)
    {
        var seg = new Segment(_mock, ax, ay, bx, by);
        var result = seg.GetQuadrant();

        Assert.Contains(shouldContain, result);
        Assert.DoesNotContain(shouldNotContain, result);
    }
}
```

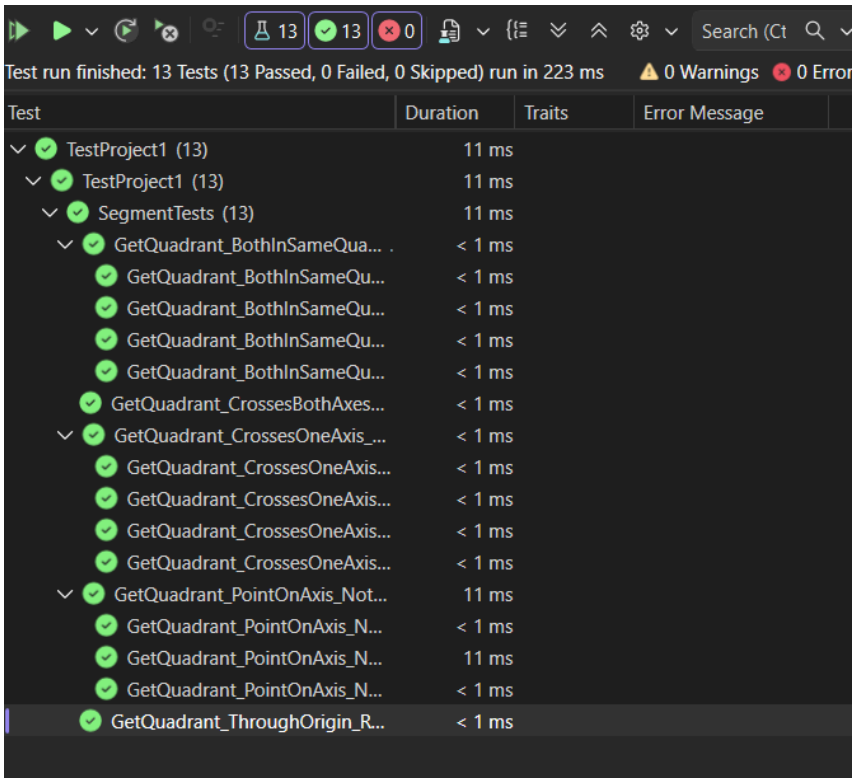


Рисунок 10 - Прохождение тестов

Тема 3. «РАБОТА С ГРАФИКОЙ»

Цель работы:

Задание 1. Разработать программу движения фигуры при помощи таймера. Регулируя интервал работы таймера, можно управлять скоростью движения фигуры. Направление движения фигуры задается с помощью изменения координат по горизонтали и (или) по вертикали.

Задание 2. Учесть, что при изменении размеров формы фигура все равно должна двигаться в заданных направлениях в пределах формы. Добавьте в проект настройку параметров: выбор цвета фигуры с помощью диалогового окна выбора цвета и настройку скорости движения фигуры. Настройка параметров должна происходить с помощью второй формы.

Задание 3. Инсталляция приложения
Добавить иконку в проект и название проекта. Для созданного вами приложения провести инсталляцию на свой компьютер. Инсталляция происходит при помощи мастера публикации (находиться в свойствах проекта).

Вариант 25

Нарисовать пульсирующий эллипс (увеличивающийся и уменьшающийся), который меняет цвет при увеличении и уменьшении и движется вверх-вниз. Программа завершается по нажатию любой клавиши.

Выполнение заданий

3.1. Код программы

Form1

```
public partial class Form1 : Form, ISettings
{
    private float _posY;
    private float _size = 40;
    private float _sizeDir = 0.5f;
    private bool _goingDown = true;
    private int _time = DateTime.Now.Millisecond;

    public int speed { get; set; } = 2;
    public Color colorStart { get; set; } = Color.Red;
    public Color colorEnd { get; set; } = Color.Blue;

    public Form1()
    {
        InitializeComponent();
        _posY = Height / 2;
    }

    private void настройкиToolStripMenuItem_Click(object sender,
EventArgs e)
    {
        Form2 form2 = new Form2(this);
        form2.Show();
    }

    public void SetSpeed(int speed)
    {
        speed = speed;
    }

    private Color SetColor() => Color.FromArgb(
            red: (int)(colorStart.R + (colorEnd.R - colorStart.R) *
(Math.Sin(_time) * 0.5 + 0.5)),
            green: (int)(colorStart.G + (colorEnd.G - colorStart.G) *
(Math.Sin(_time) * 0.5 + 0.5)),
            blue: (int)(colorStart.B + (colorEnd.B - colorStart.B) *
(Math.Sin(_time) * 0.5 + 0.5))
            );
```

```
    private void timer1_Tick(object sender, EventArgs e)
    {
        if (_goingDown)
        {
            _posY += speed;
            if (_posY ≥ ClientSize.Height - 30) _goingDown = false; //
достигли низа
        }
        else
        {
            _posY -= speed;
            if (_posY ≤ 0 + 30) _goingDown = true; // достигли верха
        }

        // Пульсация размера
        _size += _sizeDir * speed;
        if (_size ≥ 100 || _size ≤ 20) _sizeDir = -_sizeDir;

        _time = DateTime.Now.Millisecond;
        pictureBox1.Invalidate();
    }

    private void pictureBox1_Paint(object sender, PaintEventArgs e)
    {
        var g = e.Graphics;
        g.SmoothingMode =
System.Drawing.Drawing2D.SmoothingMode.AntiAlias;
        g.Clear(Color.White);

        int cx = Width / 2;

        g.FillEllipse(
            new SolidBrush(SetColor()),
            cx - _size, _posY - 30,
            _size * 2, 30
        );
    }
}
```

Form2

```
public partial class Form2 : Form
{
    ISettings _param;
    public Form2(ISettings param)
```

```
    {
        InitializeComponent();
        _param = param;
        label1.BackColor = _param.colorStart;
        label2.BackColor = _param.colorEnd;
        trackBar2.Value = _param.speed;
    }

    private void label1_Click(object sender, EventArgs e)
    {
        ColorDialog cd = new ColorDialog();
        if (cd.ShowDialog() == DialogResult.OK)
        {
            label1.BackColor = cd.Color;
            _param.colorStart = cd.Color;
        }
    }

    private void label2_Click(object sender, EventArgs e)
        {
            ColorDialog cd = new ColorDialog();
            if (cd.ShowDialog() == DialogResult.OK)
        {
            label2.BackColor = cd.Color;
            _param.colorEnd = cd.Color;
        }
    }

    private void trackBar2_ValueChanged(object sender, EventArgs e)
    {
        _param.speed = trackBar2.Value;
    }
}
```

ISettings

```
namespace UP3
{
    public interface ISettings
    {
        int speed { get; set; }
        Color colorStart { get; set; }
        Color colorEnd { get; set; }
        void SetSpeed (int speed);
    }
}
```

3.2. Демонстрация

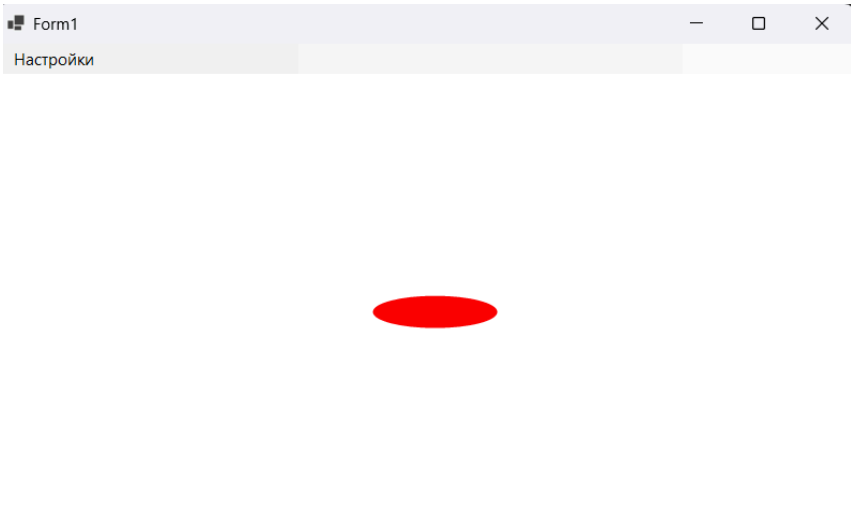


Рисунок 11 - Изменение цвета и размера

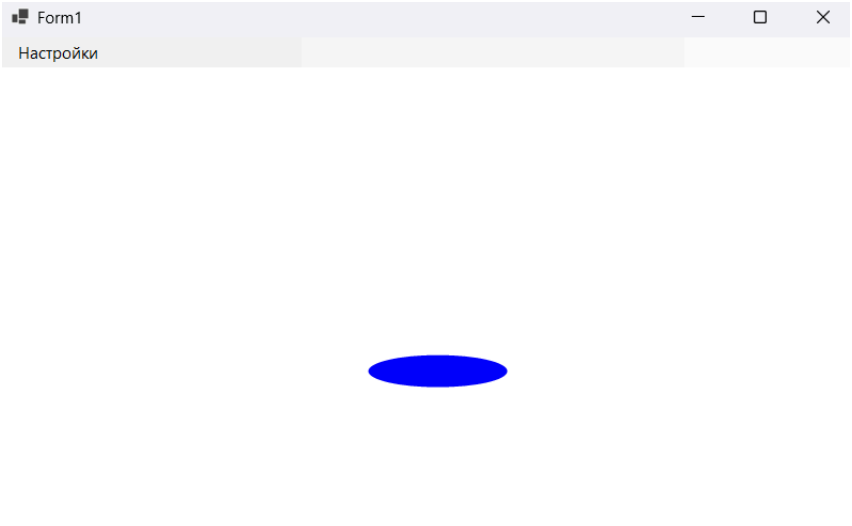


Рисунок 12 - Изменение цвета и размера

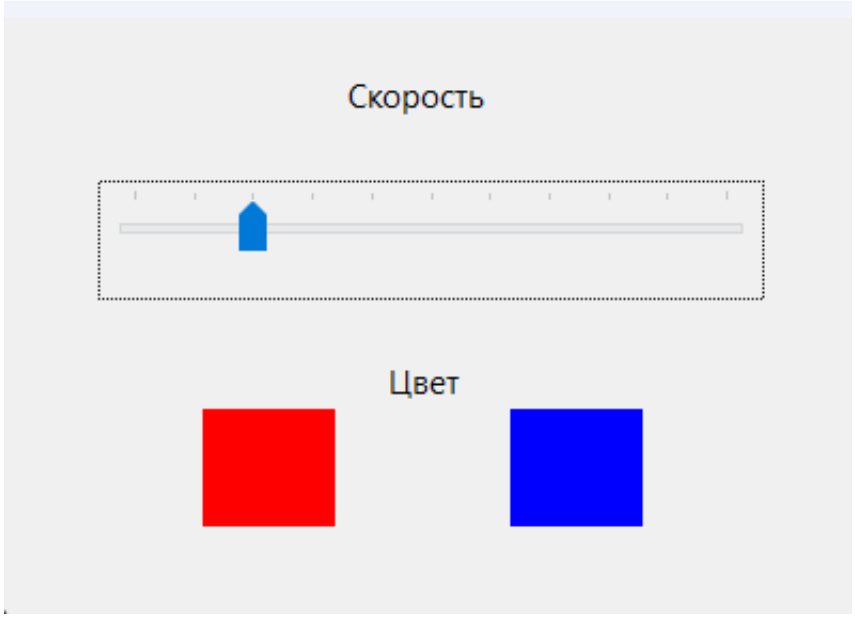


Рисунок 13 - Форма 2, изменение цвета, размера и скорости

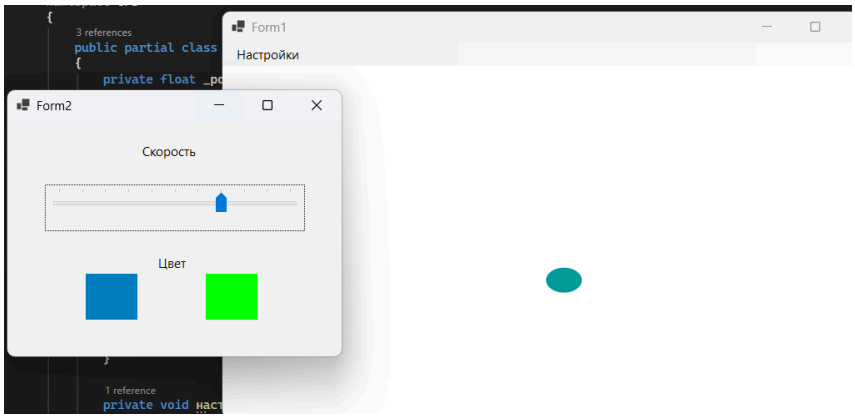


Рисунок 14 - Изменение цвета, размера и скорости

Тема 4. «ПОСТРОЕНИЕ ГРАФИКОВ И ДИАГРАММ»

Цель работы: получить практические навыки разработки проектов с использованием стандартного компонента chart для построения графиков, диаграмм, зависимостей по таблице данных.

Задание 1. Основной проект Разработать программу для ввода данных по вашему варианту задания и добавления в таблицу вводимых данных. Опрос пользователей производиться при помощи отдельной вкладки на форме, результаты опроса в виде таблицы должны отображаться на другой вкладке.

Задание 2. Сохранение и загрузка данных Добавьте возможность сохранить результаты опроса в текстовый файл и загрузить результаты опроса из текстового файла в таблицу на форме.

Задание 3. Построение диаграммы Добавьте в проект 3-ую вкладку, на которой требуется построить график или диаграмму (в зависимости от варианта). Построение диаграммы производиться по таблице данных с помощью компонента Chart.

Вариант 5

Разработать проект для ввода данных о количестве пятёрок, четверок, троек, двоек и не аттестованных по разным дисциплинам для каждой группы. Программа должна вычислять абсолютную и качественную успеваемость в процентах по каждому предмету. Абсолютная успеваемость рассчитывается как отношение суммы оценок без двоек к сумме всех оценок (в процентах), качественная успеваемость рассчитывается как сумма четвёрок и пятёрок к сумме всех оценок (в процентах). Результаты сохранять и отображать на второй вкладке в виде таблицы. Также должна быть возможность загрузить данные из текстового файла (номер группы, количество дисциплин, названия учебных дисциплин, количество пятёрок, четверок, троек, двоек и не аттестованных по разным дисциплинам) и сохранить данные в текстовый файл в таком же формате. На третьей вкладке построить столбчатую диаграмму, показывающую уровень успеваемости по разным дисциплинам (по одной группе).

Выполнение заданий

4.1. Код программы

```
public partial class Form1 : Form
{
    private string _groupName = "";
    private List<string> _disciplines = new();
    private List<int> _fives = new();
    private List<int> _fours = new();
    private List<int> _threes = new();
    private List<int> _twos = new();
    private List<int> _noAttest = new();

    private TabControl tabControl;

    // Вкладка 1 — ввод
    private TextBox tbGroup;
    private TextBox tbDiscipline;
    private NumericUpDown numFives, numFours, numThrees, numTwos,
numNoAttest;
    private Button btnAdd, btnClear;
    private ListBox lbDisciplines;

    // Вкладка 2
    private DataGridView dgv;
    private Button btnSave, btnLoad;
```

```
    // Вкладка 3
    private Chart chart;
    private Button btnDraw;

    public Form1()
    {
        InitializeComponent();
        BuildUI();
    }

    private void BuildUI()
    {
        Text = "Успеваемость по группе";
        Size = new Size(850, 600);
        StartPosition = FormStartPosition.CenterScreen;

        tabControl = new TabControl { Dock = DockStyle.Fill };
        var tab1 = new TabPage("Ввод данных");
        var tab2 = new TabPage("Таблица результатов");
        var tab3 = new TabPage("Диаграмма");

        tabControl.TabPages.Add(tab1);
        tabControl.TabPages.Add(tab2);
        tabControl.TabPages.Add(tab3);
        Controls.Add(tabControl);

        BuildTab1(tab1);
        BuildTab2(tab2);
        BuildTab3(tab3);
    }

    private void BuildTab1(TabPage tab)
    {
        var lblGroup = new Label { Text = "Номер группы:", Location =
new Point(10, 15), AutoSize = true };
        tbGroup = new TextBox { Location = new Point(130, 12), Width =
150 };

        var lblDisc = new Label { Text = "Дисциплина:", Location = new
Point(10, 50), AutoSize = true };
        tbDiscipline = new TextBox { Location = new Point(130, 47),
Width = 200 };

        int y = 85;
        var labels = new[] { "Пятёрки:", "Четвёрки:", "Тройки:",
```

```
"Двойки:", "Не аттест.:" };
        numFives = new NumericUpDown { Location = new Point(130, y),
Width = 70, Minimum = 0, Maximum = 999 };
        numFours = new NumericUpDown { Location = new Point(130, y +
35), Width = 70, Minimum = 0, Maximum = 999 };
        numThrees = new NumericUpDown { Location = new Point(130, y +
70), Width = 70, Minimum = 0, Maximum = 999 };
        numTwos = new NumericUpDown { Location = new Point(130, y +
105), Width = 70, Minimum = 0, Maximum = 999 };
        numNoAttest = new NumericUpDown { Location = new Point(130, y +
140), Width = 70, Minimum = 0, Maximum = 999 };

        for (int i = 0; i < labels.Length; i++)
            tab.Controls.Add(new Label
            {
                Text = labels[i],
                Location = new Point(10, y + i * 35),
                AutoSize = true
            });

        btnAdd = new Button { Text = "Добавить дисциплину", Location =
new Point(10, 310), Width = 160, Height = 30 };
        btnClear = new Button { Text = "Очистить всё", Location = new
Point(180, 310), Width = 160, Height = 30 };
        btnAdd.Click += BtnAdd_Click;
        btnClear.Click += BtnClear_Click;

        var lblList = new Label { Text = "Добавленные дисциплины:",
Location = new Point(350, 10), AutoSize = true };
        lbDisciplines = new ListBox { Location = new Point(350, 35),
Size = new Size(450, 320) };

        tab.Controls.AddRange(new Control[]
        {
            lblGroup, tbGroup,
            lblDisc, tbDiscipline,
            numFives, numFours, numThrees, numTwos, numNoAttest,
            btnAdd, btnClear,
            lblList, lbDisciplines
        });
    }

    private void BuildTab2(TabPage tab)
    {
        dgv = new DataGridView
        {
```

```
            Location = new Point(10, 10),
            Size = new Size(800, 430),
            ReadOnly = true,
            AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill
        };

        dgv.Columns.Add("disc", "Дисциплина");
        dgv.Columns.Add("fives", "5");
        dgv.Columns.Add("fours", "4");
        dgv.Columns.Add("threes", "3");
        dgv.Columns.Add("twos", "2");
        dgv.Columns.Add("noatt", "Не атт.");
        dgv.Columns.Add("total", "Всего");
        dgv.Columns.Add("abs", "Абс. усп. %");
        dgv.Columns.Add("qual", "Кач. усп. %");

        btnSave = new Button { Text = "Сохранить в файл", Location =
new Point(10, 455), Width = 160, Height = 30};
        btnLoad = new Button { Text = "Загрузить из файла", Location =
new Point(180, 455), Width = 160, Height = 30  };
        btnSave.Click += BtnSave_Click;
        btnLoad.Click += BtnLoad_Click;

        tab.Controls.AddRange(new Control[] { dgv, btnSave, btnLoad });
    }

    private void BuildTab3(TabPage tab)
    {
        chart = new Chart { Location = new Point(10, 50), Size = new
Size(800, 430) };
        chart.ChartAreas.Add(new ChartArea("Main"));
        chart.Legends.Add(new Legend("Main"));

        btnDraw = new Button { Text = "Построить диаграмму", Location =
new Point(10, 10), Width = 180, Height = 30};
        btnDraw.Click += BtnDraw_Click;

        tab.Controls.AddRange(new Control[] { btnDraw, chart });
    }

    private void BtnAdd_Click(object sender, EventArgs e)
    {
        if (string.IsNullOrWhiteSpace(tbGroup.Text))
        {
            MessageBox.Show("Введите номер группы!");
            return;
```

```
        }
        if (string.IsNullOrWhiteSpace(tbDiscipline.Text))
        {
            MessageBox.Show("Введите название дисциплины!");
            return;
        }

        _groupName = tbGroup.Text.Trim();
        _disciplines.Add(tbDiscipline.Text.Trim());
        _fives.Add((int)numFives.Value);
        _fours.Add((int)numFours.Value);
        _threes.Add((int)numThrees.Value);
        _twos.Add((int)numTwos.Value);
        _noAttest.Add((int)numNoAttest.Value);

        // Обновить список и таблицу
        RefreshListBox();
        RefreshTable();

        // Очистить поля ввода
        tbDiscipline.Clear();
        numFives.Value = numFours.Value = numThrees.Value =
        numTwos.Value = numNoAttest.Value = 0;
    }

    private void BtnClear_Click(object sender, EventArgs e)
    {
        if (MessageBox.Show("Очистить все данные?", "Подтверждение",
            MessageBoxButtons.YesNo) != DialogResult.Yes) return;

        _groupName = "";
        _disciplines.Clear();
        _fives.Clear(); _fours.Clear(); _threes.Clear();
        _twos.Clear(); _noAttest.Clear();

        tbGroup.Clear();
        tbDiscipline.Clear();
        numFives.Value = numFours.Value = numThrees.Value =
        numTwos.Value = numNoAttest.Value = 0;

        RefreshListBox();
        RefreshTable();
    }

    private void BtnSave_Click(object sender, EventArgs e)
    {
```

```
        if (_disciplines.Count == 0)
        {
            MessageBox.Show("Нет данных для сохранения!");
            return;
        }

        using var dialog = new SaveFileDialog
        {
            Filter = "Текстовые файлы|*.txt",
            FileName = $"group_{_groupName}.txt"
        };

        if (dialog.ShowDialog() != DialogResult.OK) return;

        using var writer = new StreamWriter(dialog.FileName);
        writer.WriteLine(_groupName);
        writer.WriteLine(_disciplines.Count);

        for (int i = 0; i < _disciplines.Count; i++)
        {
            writer.WriteLine(_disciplines[i]);
            writer.WriteLine(_fives[i]);
            writer.WriteLine(_fours[i]);
            writer.WriteLine(_threes[i]);
            writer.WriteLine(_twos[i]);
            writer.WriteLine(_noAttest[i]);
        }

        MessageBox.Show("Данные сохранены!");
    }

    private void BtnLoad_Click(object sender, EventArgs e)
    {
        using var dialog = new OpenFileDialog { Filter = "Текстовые
файлы|*.txt" };
        if (dialog.ShowDialog() != DialogResult.OK) return;

        try
        {
            using var reader = new StreamReader(dialog.FileName);

            _groupName = reader.ReadLine() ?? "";
            int count = int.Parse(reader.ReadLine() ?? "0");

            // Очищаем старые данные
            _disciplines.Clear();
```

```
            _fives.Clear(); _fours.Clear(); _threes.Clear();
            _twos.Clear(); _noAttest.Clear();

            for (int i = 0; i < count; i++)
            {
                _disciplines.Add(reader.ReadLine() ?? "");
                _fives.Add(int.Parse(reader.ReadLine() ?? "0"));
                _fours.Add(int.Parse(reader.ReadLine() ?? "0"));
                _threes.Add(int.Parse(reader.ReadLine() ?? "0"));
                _twos.Add(int.Parse(reader.ReadLine() ?? "0"));
                _noAttest.Add(int.Parse(reader.ReadLine() ?? "0"));
            }

            tbGroup.Text = _groupName;
            RefreshListBox();
            RefreshTable();

            MessageBox.Show("Данные загружены!");
        }
        catch (Exception ex)
        {
            MessageBox.Show($"Ошибка загрузки: {ex.Message}");
        }
    }

    private void BtnDraw_Click(object sender, EventArgs e)
    {
        if (_disciplines.Count == 0)
        {
            MessageBox.Show("Нет данных для диаграммы!");
            return;
        }

        DrawDiagram();
    }

    private void RefreshListBox()
    {
        lbDisciplines.Items.Clear();
        for (int i = 0; i < _disciplines.Count; i++)
        {
            int total = Total(i);
            lbDisciplines.Items.Add(
                $"{_disciplines[i]} | 5:{_fives[i]} 4:{_fours[i]} " +
                $"3:{_threes[i]} 2:{_twos[i]} н/а:{_noAttest[i]} |
Всего:{total}"
```

```
            );
        }
    }
    private void RefreshTable()
    {
        dgv.Rows.Clear();

        for (int i = 0; i < _disciplines.Count; i++)
        {
            int total = Total(i);
            double abs = AbsPerf(i);
            double qual = QualPerf(i);

            dgv.Rows.Add(
                _disciplines[i],
                _fives[i],
                _fours[i],
                _threes[i],
                _twos[i],
                _noAttest[i],
                total,
                $"{abs:F1}%",
                $"{qual:F1}%"
            );
        }
    }

    private void DrawDiagram()
    {
        chart.Series.Clear();
        chart.Titles.Clear();
        chart.Titles.Add($"Успеваемость группы {_groupName}");
        chart.Titles[0].Font = new Font("Arial", 14, FontStyle.Bold);

        var serAbs = new Series("Абсолютная успеваемость")
        {
            ChartType = SeriesChartType.Column,
            ChartArea = "Main",
            Color = Color.SteelBlue,
            IsValueShownAsLabel = true,
            LabelFormat = "F1"
        };

        var serQual = new Series("Качественная успеваемость")
        {
            ChartType = SeriesChartType.Column,
```

```
            ChartArea = "Main",
            Color = Color.OrangeRed,
            IsValueShownAsLabel = true,
            LabelFormat = "F1"
        };

        for (int i = 0; i < _disciplines.Count; i++)
        {
            serAbs.Points.AddXY(_disciplines[i], AbsPerf(i));
            serQual.Points.AddXY(_disciplines[i], QualPerf(i));
        }

        chart.Series.Add(serAbs);
        chart.Series.Add(serQual);

        var area = chart.ChartAreas["Main"];
        area.AxisY.Minimum = 0;
        area.AxisY.Maximum = 100;
        area.AxisY.Title = "Успеваемость %";
        area.AxisX.Title = "Дисциплина";
        area.AxisX.LabelStyle.Angle = -30;
    }

    // Всего оценок по дисциплине
    private int Total(int i)
        => _fives[i] + _fours[i] + _threes[i] + _twos[i] +
_noAttest[i];

    // Абсолютная успеваемость
    private double AbsPerf(int i)
    {
        int total = Total(i);
        if (total == 0) return 0;
        int withoutBad = _fives[i] + _fours[i] + _threes[i];
        return withoutBad / (double)total * 100;
    }

    // Качественная успеваемость
    private double QualPerf(int i)
    {
        int total = Total(i);
        if (total == 0) return 0;
        int good = _fives[i] + _fours[i];
        return good / (double)total * 100;
    }
}
```

4.2. Демонстрация

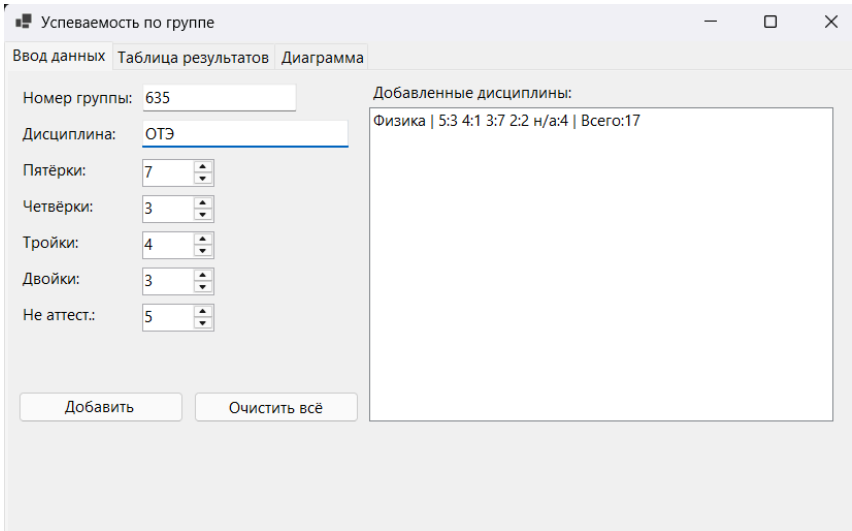
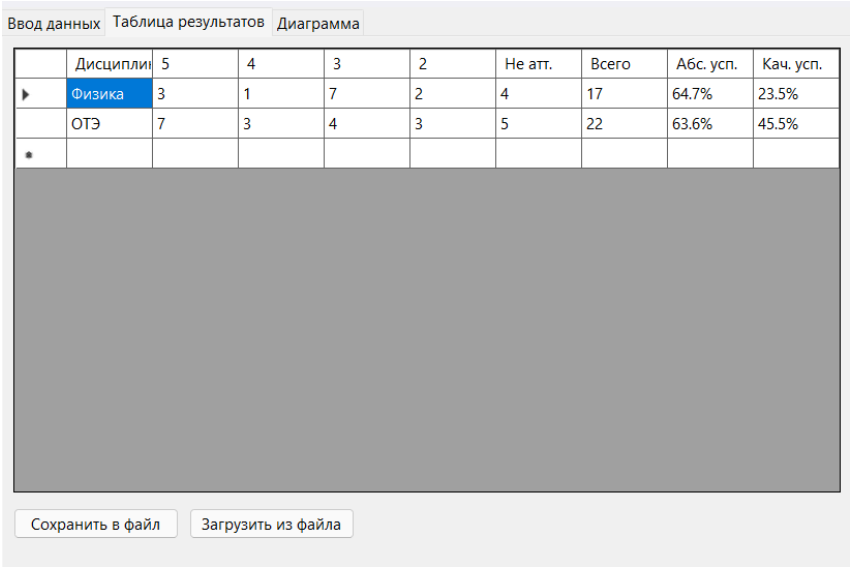


Рисунок 15 - Добавление групп



	Дисципли	5	4	3	2	Не атт.	Всего	Абс. усп.	Кач. усп.
▶	Физика	3	1	7	2	4	17	64.7%	23.5%
	ОТЭ	7	3	4	3	5	22	63.6%	45.5%
*									

Рисунок 16 - Занисение в таблицу с возможностью сохранения

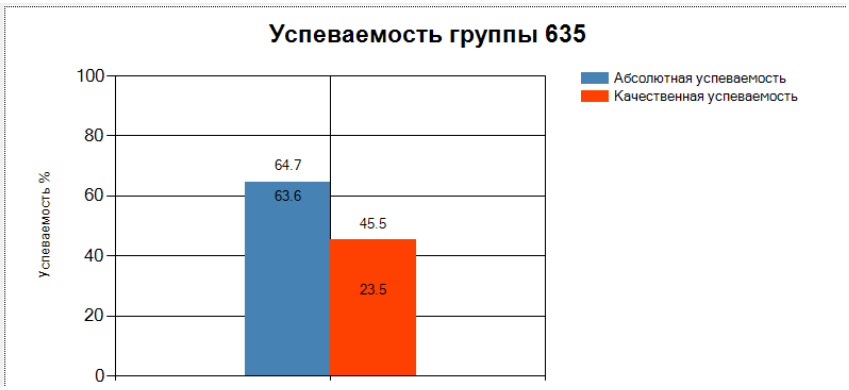


Рисунок 17 - Построение графика по группе

Тема 5. «ПРОГРАММЫ ДЛЯ ОБРАБОТКИ ТЕКСТОВЫХ ФАЙЛОВ»

Цель работы: получить практические навыки создания и подключения собственной динамической библиотеки, разработки алгоритмов сложного поиска и методов по обработке текстовых файлов.

Задание 1. Разработать динамическую библиотеку работы со словарем. Словарь (примерно 90 000 слов) хранится в текстовом файле, каждое слово на отдельной строке. В библиотеке необходимо реализовать следующие функции:

- загрузку словаря в динамический список;
- добавление слов в словарь, причем если добавляемое слово уже есть в словаре, оно не должно добавляться повторно;
- поиск слов в списке по своему заданию и формирование нового списка с результатами;
- нечёткий поиск слов с расстоянием Левенштейна не более 3 (то есть слов, включающих все заданные буквы в том же порядке или с вставкой/ удалением/заменой, но не более 3-х символов);
- В случае, если поиск не дал результатов, методы должны каким-либо образом информировать об этом.

Задание 2. Протестируйте собственную функцию поиска с помощью юнит-тестов. Необходимо учесть ситуацию, что поиск может не дать результатов.

Задание 3. Создайте основной проект, подключите собственную библиотеку и разработайте удобный пользовательский интерфейс, обеспечивающий.

Вариант 5

Найти в словаре слова, включающие все буквы введенного слова плюс любые другие. Результаты сохранить в отдельный текстовый файл.

Выполнение заданий

5.1. Код программы

5.2. Демонстрация

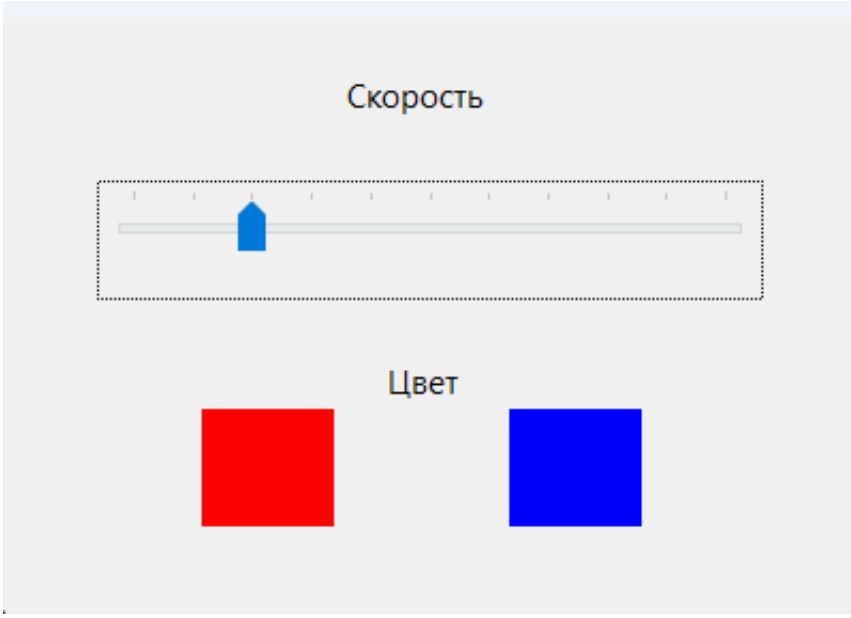


Рисунок 18 - Форма 2, изменение цвета, размера и скорости

Тема 6. «РАЗРАБОТКА ИГРОВОГО ПРИЛОЖЕНИЯ НА ОСНОВЕ XML-ФАЙЛА»

Цель работы: получить практические навыки проектирования многомодульных приложений и освоить методы обработки XML-файла.

Задание 4. Требуется создать программу компьютерного тестирования, которая получает список вопросов, изображений к ним и вариантов ответов из XML-файла специального формата. В файле находятся тестовые задания по нескольким темам. Внутри конкретной темы имеется 3 уровня сложности. В каждом уровне сложности имеются по 10 заданий или вопросов с вариантами правильных и неправильных ответов. Предполагается, что пользователь может выбрать тему для тестирования и последовательно проходить задания из данной темы по уровням сложности. Переход на следующий уровень сложности в данной теме осуществляется строго после того, как игрок наберет не менее 80 баллов за последнее тестирование (по 100-балльной системе). Вопросы и варианты ответов из текущего уровня сложности должны каждый раз выдаваться в случайном порядке. Вопросов в уровне может быть больше, чем будет выдаваться на экран в течение сеанса игры (например, вопросов в уровне 10, а выдается 5 случайных за один сеанс игры). Необходимо также ограничить время сеанса игры и обеспечить возврат в главное меню программы.

Задание 5. Добавьте в проект панель администратора, в которой можно быстро и удобно создавать новую тему (с тремя уровнями сложности), добавлять вопросы, изображения к ним и варианты ответов в новую или в уже существующую тему. Вопросы и варианты ответов записываются в XML-файл самой программой.

Вариант 5

Изучение программирования. Создаются различные темы, например: «Операции», «Ветвления», «Циклы», «Объектно-ориентированное программирование» и т.п. Программа предлагает вопросы с вариантами ответов по выбранной теме. Вопрос может сопровождаться картинкой. На некоторые вопросы может быть несколько правильных вариантов ответов.

Выполнение заданий

6.1. Сценарий использования программы

1. Разработать диаграмму прецедентов. Рассмотрите роли и варианты использования, которые будет предоставлять программа при работе с ней.
2. Подготовить макеты основных окон проекта.
3. Подготовить XML-файл специального формата для тестирования работы программы.
4. Найти необходимые изображения или иные ресурсы для проекта и поместить их во вложенную папку проекта.
5. Разработать программный код каждого модуля, прокомментировать назначение переменных и основных методов.
6. Выполнить отладку программных модулей с применением автоматизированных отладочных средств.
7. Подготовить тест-кейсы (тестовые случаи) по шаблону.
8. Провести испытания программы в соответствие с тест-кейсами.
9. Оформить отчет по практической работе, снабдить его скриншотами.

6.2. Код программы

```
public partial class Form1 : Form
{
    private string _xmlPath = ".\\test.xml";
    private string _topic = "Циклы";
    private int _level = 1;

    // Вопросы текущей сессии
    private List<(string Text, List<(string Text, bool Right)>
Answers)> _questions = new();
    private int _current = 0;
    private int _score = 0;


    private Button[] _answerButtons = null!;

    public Form1()
    {
```

```
        InitializeComponent();

        _answerButtons = new[] { button1, button2, button3, button7 };


        SetGameVisible(false);
        label3.Text = _level.ToString();
    }

    private void button6_Click(object sender, EventArgs e)
    {
        if (!File.Exists(_xmlPath))
        {
            MessageBox.Show("Файл test.xml не найден!");
            return;
        }

        LoadQuestions();

        if (_questions.Count == 0)
        {
            MessageBox.Show($"Нет вопросов для уровня {_level}");
            return;
        }

        _current = 0;
        _score = 0;
        button8.Enabled = false;

        SetGameVisible(true);
        button6.Visible = false;

        ShowQuestion();
    }

    private void button4_Click(object sender, EventArgs e)
    {
        if (_current > 0)
        {
            _score -= 20;
            _current--;
            ShowQuestion();
        }
    }

    private void button5_Click(object sender, EventArgs e)
```

```
    {
        if (_current < _questions.Count - 1)
        {
            if (IsRichtige()) _score += 20;
            _current++;
            ShowQuestion();
        }
        else
        {
            ShowResult();
        }
    }

    private void button1_Click(object sender, EventArgs e) ⇒
SelectAnswer(button1);
    private void button2_Click(object sender, EventArgs e) ⇒
SelectAnswer(button2);
    private void button3_Click(object sender, EventArgs e) ⇒
SelectAnswer(button3);
    private void button7_Click(object sender, EventArgs e) ⇒
SelectAnswer(button7);

    private void SelectAnswer(Button clicked)
    {
        if (clicked.BackColor == Color.LightGreen)
            clicked.BackColor = SystemColors.Control;
        else
            clicked.BackColor = Color.LightGreen;
    }

    private void LoadQuestions()
    {
        _questions.Clear();
        var doc = XDocument.Load(_xmlPath);

        var levelEl = doc.Root?
            .Element("topics")?
            .Elements("topic")
            .FirstOrDefault(t ⇒ t.Attribute("name")?.Value == _topic)?
            .Elements("level")
            .FirstOrDefault(l ⇒ l.Attribute("difficulty")?.Value ==
_level.ToString());

        if (levelEl == null) return;

        var all = levelEl.Elements("q").Select(q ⇒
```

```
        {
            var text = q.Attribute("text")?.Value ?? "";
            var answers = q.Elements("a")
                .Select(a => (a.Value, a.Attribute("right")?.Value ==
"yes"))
                .ToList();
            return (text, answers);
        }).ToList();

        _questions = all.OrderBy(_ =>
Random.Shared.Next()).Take(5).ToList();
    }

    private void ShowQuestion()
    {
        var (text, answers) = _questions[_current];

        label2.Text = $"[{_current + 1}/{_questions.Count}]  {text}";

        foreach (var btn in _answerButtons)
        {
            btn.BackColor = SystemColors.Control;
            btn.Visible = false;
            btn.Text = "";
        }

        var shuffled = answers.OrderBy(_ =>
Random.Shared.Next()).ToList();
        for (int i = 0; i < Math.Min(shuffled.Count, 4); i++)
        {
            _answerButtons[i].Text = shuffled[i].Text;
            _answerButtons[i].Tag = shuffled[i].Right;
            _answerButtons[i].Visible = true;
        }

        button4.Enabled = _current > 0;
        button5.Text = _current == _questions.Count - 1
            ? "Завершить" : "⟶";
    }

    private bool IsRichtige()
    {
        var correctSet = _answerButtons
            .Where(b => b.Visible && (bool)(b.Tag ?? false))
            .Select(b => b.Text).ToHashSet();
```

```
        var selectedSet = _answerButtons
            .Where(b => b.Visible && b.BackColor == Color.LightGreen)
            .Select(b => b.Text).ToHashSet();
        return correctSet.SetEquals(selectedSet);
    }
    private void ShowResult()
    {
        if (IsRichtige()) _score += 20;

        bool passed = _score >= 80;

        MessageBox.Show(
            $"Тест завершён!\nСчёт: {_score}/100\n" +
            (passed ? "Уровень пройден!" : "Попробуй ещё раз"),
            "Результат"
        );
        button8.Enabled = passed && _level < 3;
        button6.Visible = true;
        SetGameVisible(false);
    }

    private void button8_Click(object sender, EventArgs e)
    {
        _level++;
        label3.Text = _level.ToString();
        button8.Enabled = false;
    }

    private void SetGameVisible(bool visible)
    {
        label2.Visible = visible;
        label4.Visible = visible;
        button4.Visible = visible;
        button5.Visible = visible;
        foreach (var btn in _answerButtons ?? Array.Empty<Button>())
            btn.Visible = visible;
    }
}
```

6.3. Демонстрация

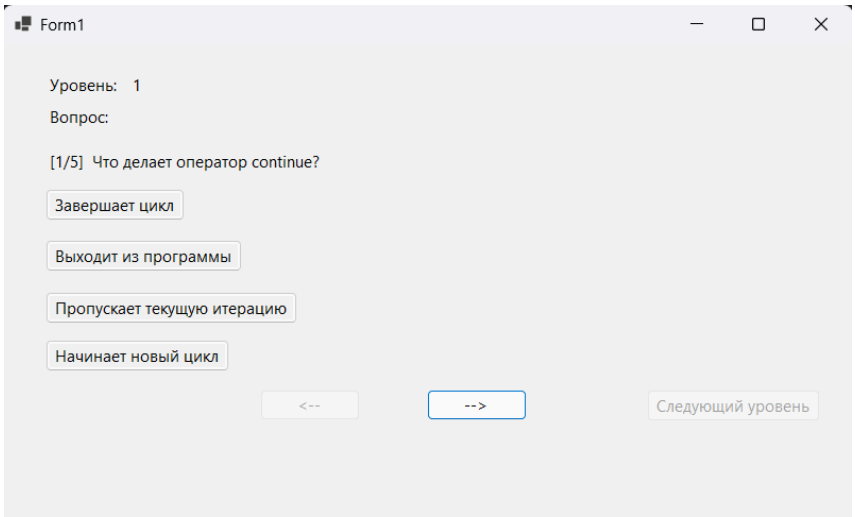


Рисунок 19 - Выбор вопросов

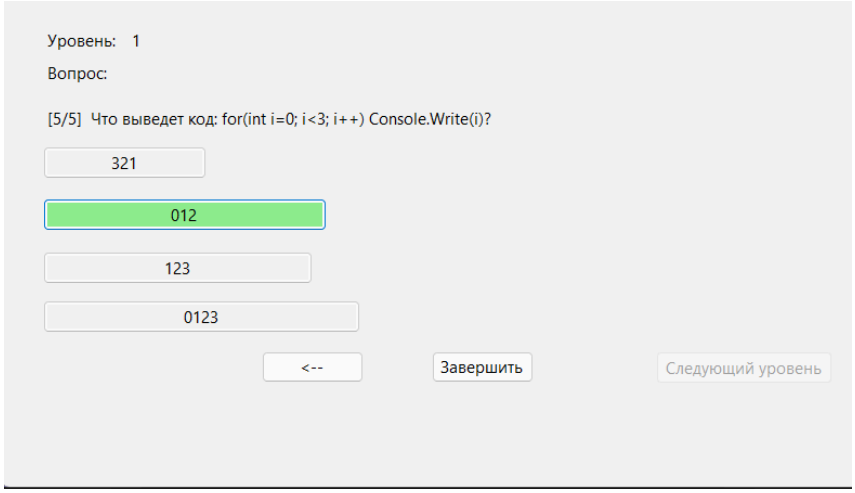


Рисунок 20 - Выбор вопросов

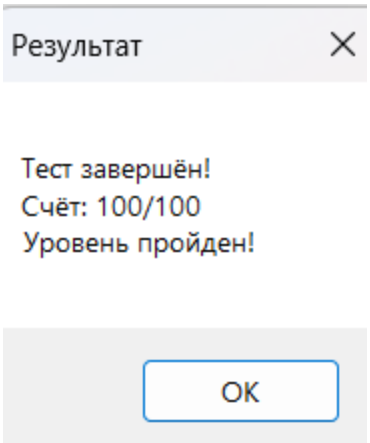


Рисунок 21 - Завершение работы

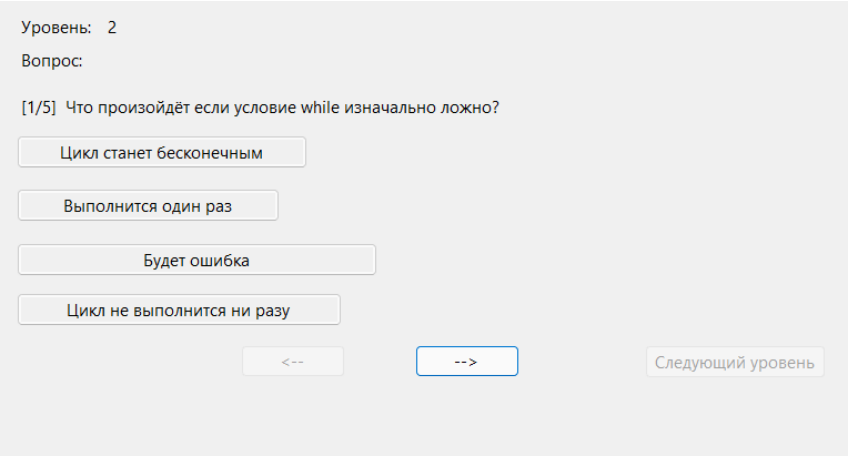


Рисунок 22 - Переход на следующий уровень

Тема 7. «ВЕКТОРНЫЙ РЕДАКТОЕ»

Цель работы: получить практические навыки разработки иерархии классов сложных проектов, получить опыт применения команд графики и работы с бинарными файлами объектов данных для сохранения и загрузки построенных векторных изображений.

Задание 6. Основные классы проекта

- Разработать базовый класс «Фигура», позволяющий прорисовывать некую фигуру в заданной прямоугольной области, сдвигать фигуру по горизонтали и вертикали на 1 и на 5 пикселов, перемещать фигуру в заданную точку;
- Разработать производные классы фигур по индивидуальному заданию, все фигуры строятся относительно базовой точки (верхнего левого угла прямоугольной области);
- Разработать класс для контура фигуры (основные данные: толщина контура и его цвет);
- Разработать класс для сохранения последних изменений полотна;
- Разработать класс обрамления фигуры при её выделении (прорисовки маркеров по краям прямоугольной области фигуры).

Задание 7.

Вариант 5 Круг, эллипс, сектор круга, половина круга, срезанный круг Заливка фигуры + заливка фигуры **Выполнение заданий**

7.1. Математическая модель

Дано:

Найти:

Связь:

ОДЗ:

7.2. Проектирование пользовательского интерфейса

7.3. Свойства компонентов формы

Таблица 5 -

Название компонента	Свойства	Значения

7.4. Сценарий использования программы

1. что происходит

7.5. Код программы

7.6. Схемы основных подпрограмм

Блок-схема 10 —

7.7. Протокол испытаний

7.8. Тестирование проекта:

7.9. Тестирование проекта:

Тема 8. «РАЗРАБОТКА ИГРЫ С ЭЛЕМЕНТАМИ ГРАФИКИ»

Цель работы:

Задание 8. Разработать игру с графическим интерфейсом. Проект должен включать меню с пунктами Файл (если он требуется), Настройки и Справка.

Задание 9. Добавьте в проект настройку параметров игры: времени сеанса игры или уровня сложности игры, цветов некоторых элементов игры, выбора персонажа и др. Добавьте в проект возможность авторизации (ввода логина) и сохранения результатов игры в двоичный файл. Просмотр предыдущих результатов игры конкретного игрока организуйте при выборе одного из пунктов меню.

Вариант 5

Игра «Прожорливая гусеница». Игра развивает память и внимание. По игровому полю движется гусеница, которой можно управлять с помощью клавиш со стрелками. На поле периодически появляются и исчезают красные и зеленые яблоки. Их надо успеть «проглотить», приблизившись на близкое расстояние. При этом игрок должен в уме вести счет, сколько красных яблок съедено и сколько зелёных. Игра заканчивается через

определенное время. После окончания сеанса игры пользователь вводит результаты своих подсчетов (сколько красных яблок съедено и сколько зелёных). Выигрыш в том случае, если съедено не менее определенного количества яблок и если подсчеты игрока верны. Программа демонстрирует работу с графикой.

Выполнение заданий

8.1. Код программы

Form1

```
public partial class Form1 : Form, ISnake
{
    private Snake _snake;
    private int _gameSeconds = 59;
    private int _tickCount = 0;
    private bool _gameRunning = false;

    private const int TICKS_PER_SEC = 50;

    public Form1()
    {
        InitializeComponent();
        _snake = new Snake(this);

        KeyDown += Form1_KeyDown;
        KeyPreview = true;
        button1.Click += button1_Click;

        pictureBox1.Visible = false;
        pictureBox2.Visible = false;
        pictureBox3.Visible = false;
        timer1.Enabled = false;
        UpdateTimerLabel();
    }

    private void button1_Click(object? sender, EventArgs e)
    {
        _snake = new Snake(this);
        _tickCount = 0;
        _gameRunning = true;

        pictureBox1.Location = new Point(100, 100);
        pictureBox1.Visible = true;
        pictureBox2.Visible = false;
        pictureBox3.Visible = false;
        UpdateTimerLabel();
```

```
        timer1.Enabled = true;
        button1.Visible = false;
        pictureBox1.Focus();
    }

    private void timer1_Tick(object? sender, EventArgs e)
    {
        if (!_gameRunning) return;

        _snake.Step(this.ClientSize.Height, this.ClientSize.Width);
        pictureBox1.Location = new Point((int)_snake.local.X,
(int)_snake.local.Y);

        _tickCount++;
        if (_tickCount % TICKS_PER_SEC == 0)
        {
            _gameSeconds--;
            UpdateTimerLabel();

            if (_gameSeconds <= 0)
                EndGame();
        }
    }
    private void Form1_KeyDown(object? sender, KeyEventArgs e)
    {
        if (!_gameRunning) return;

        switch (e.KeyCode)
        {
            case Keys.Right: _snake.SetDirectionRight(pictureBox1);
break;
            case Keys.Left: _snake.SetDirectionLeft(pictureBox1);
break;
            case Keys.Down: _snake.SetDirectionDown(pictureBox1);
break;
            case Keys.Up: _snake.SetDirectionUp(pictureBox1); break;
        }
        e.Handled = true;
    }
    private void UpdateTimerLabel()
    {
        labelTimer.Text = $"{_gameSeconds} сек";
    }

    private void EndGame()
```

```
    {
        _gameRunning = false;
        timer1.Enabled = false;
        pictureBox1.Visible = false;
        pictureBox2.Visible = false;
        pictureBox3.Visible = false;

        var actual = _snake.Eaten;
        var dialog = new ResultForm();

        if (dialog.ShowDialog(this) == DialogResult.OK)
        {
            int guessR = dialog.GuessR;
            int guessG = dialog.GuessG;

            bool correctR = guessR == actual.R;
            bool correctG = guessG == actual.G;
            bool win = correctR && correctG
                            && (actual.R + actual.G) ≥ 5;

            string result = win ? "Победа!" : "Проигрыш.";

            MessageBox.Show(
                $"{result}\n\n" +
                $"Красных: реально {actual.R}, вы сказали {guessR}\n" +
                $"Зелёных: реально {actual.G}, вы сказали {guessG}",
                "Итог игры",
                MessageBoxButtons.OK,
                win ? MessageBoxIcon.Information :
MessageBoxIcon.Warning
            );
        }

        button1.Visible = true;
        _gameSeconds = 59;
        UpdateTimerLabel();
    }
    public void SetRApplePos(int x, int y) ⇒
        pictureBox2.Location = new Point(x, y);

    public void SetGApplePos(int x, int y) ⇒
        pictureBox3.Location = new Point(x, y);

    public void SetRAppleVisible(bool visible) ⇒
        pictureBox2.Visible = visible;
```

```
    public void SetGAppleVisible(bool visible) =>
        pictureBox3.Visible = visible;
}
```

Snake

```
internal class Snake
{
    public record struct Point(float X, float Y);
    public record struct Apples(int G, int R);

    ISnake _form;

    public Point local;
    Point _nextLocal, _gApple, _rApple;
    float _speed;
    public Apples Eaten;

    const int DS = 35;
    const int DA = 20;
    const int CATCH_DIST = DA + DS;

    Random _random = new Random();

    private int _rAppleTimer = 0;
    private int _gAppleTimer = 0;
    private int _rAppleDelay = 0;
    private int _gAppleDelay = 0;

    public bool RAppleVisible { get; private set; } = false;
    public bool GAppleVisible { get; private set; } = false;

    public Point RApplePos => _rApple;
    public Point GApplePos => _gApple;

    public Snake(ISnake form)
    {
        _form = form;
        local = new Point(100, 100);
        _nextLocal = new Point(-7f, 0f);
        _speed = 7f;
    }

    public void SetDirectionRight(PictureBox pict) { _nextLocal = new
Point(_speed, 0); pict.Image = Properties.Resources.Right; }
    public void SetDirectionLeft(PictureBox pict) { _nextLocal = new
```

```
Point(-_speed, 0); pict.Image = Properties.Resources.Left; }
    public void SetDirectionDown(PictureBox pict) { _nextLocal = new
Point(0, _speed); pict.Image = Properties.Resources.Bottom; }
    public void SetDirectionUp(PictureBox pict) { _nextLocal = new
Point(0, -_speed); pict.Image = Properties.Resources.Top; }



    public void Step(float h, float w)
    {
        local = new Point(local.X + _nextLocal.X, local.Y +
_nextLocal.Y);

        if (local.X > w) local = new Point(0, local.Y);
        if (local.X < -DS) local = new Point(w, local.Y);
        if (local.Y > h) local = new Point(local.X, 0);
        if (local.Y < -DS) local = new Point(local.X, h);

        UpdateAppleTimers(h, w);

        CheckEating();
    }

    private void UpdateAppleTimers(float h, float w)
    {
        if (RAppleVisible)
        {
            _rAppleTimer--;
            if (_rAppleTimer <= 0)
            {
                RAppleVisible = false;
                _rAppleDelay = _random.Next(30, 100);
                _form.SetRAppleVisible(false);
            }
        }
        else
        {
            _rAppleDelay--;
            if (_rAppleDelay <= 0)
            {
                _rApple = RandomPos(h, w);
                RAppleVisible = true;
                _rAppleTimer = _random.Next(100, 250);
                _form.SetRApplePos((int)_rApple.X, (int)_rApple.Y);
                _form.SetRAppleVisible(true);
            }
```

```
        }

        if (GAppleVisible)
        {
            _gAppleTimer--;
            if (_gAppleTimer ≤ 0)
            {
                GAppleVisible = false;
                _gAppleDelay = _random.Next(50, 120);
                _form.SetGAppleVisible(false);
            }
        }
        else
        {
            _gAppleDelay--;
            if (_gAppleDelay ≤ 0)
            {
                _gApple = RandomPos(h, w);
                GAppleVisible = true;
                _gAppleTimer = _random.Next(80, 200);
                _form.SetGApplePos((int)_gApple.X, (int)_gApple.Y);
                _form.SetGAppleVisible(true);
            }
        }
    }

    private void CheckEating()
    {
        if (RAppleVisible)
        {
            float dx = Math.Abs(_rApple.X - local.X);
            float dy = Math.Abs(_rApple.Y - local.Y);
            if (dx ≤ CATCH_DIST && dy ≤ CATCH_DIST)
            {
                Eaten = new Apples(Eaten.G, Eaten.R + 1);
                RAppleVisible = false;
                _rAppleDelay = _random.Next(30, 80);
                _form.SetRAppleVisible(false);
            }
        }

        if (GAppleVisible)
        {
            float dx = Math.Abs(_gApple.X - local.X);
            float dy = Math.Abs(_gApple.Y - local.Y);
            if (dx ≤ CATCH_DIST && dy ≤ CATCH_DIST)
```

```
            {
                Eaten = new Apples(Eaten.G + 1, Eaten.R);
                GAppleVisible = false;
                _gAppleDelay = _random.Next(50, 100);
                _form.SetGAppleVisible(false);
            }
        }
    }

    private Point RandomPos(float h, float w)
    {
        return new Point(
            _random.Next(DA, (int)w - DA),
            _random.Next(DA + 50, (int)h - DA)
        );
    }
}
```

ISnake

```
internal interface ISnake
{
    void SetRApplePos(int x, int y);
    void SetGApplePos(int x, int y);
    void SetRAppleVisible(bool visible);
    void SetGAppleVisible(bool visible);
}
```

ResultForm

```
public partial class ResultForm : Form
{
    public int GuessR { get; private set; }
    public int GuessG { get; private set; }

    public ResultForm()
    {
        InitializeComponent();
    }

    private void btnOk_Click(object sender, EventArgs e)
    {
        GuessR = (int)nudR.Value;
        GuessG = (int)nudG.Value;
        DialogResult = DialogResult.OK;
        Close();
```

```
    }
}
```

8.2. Демонстрация

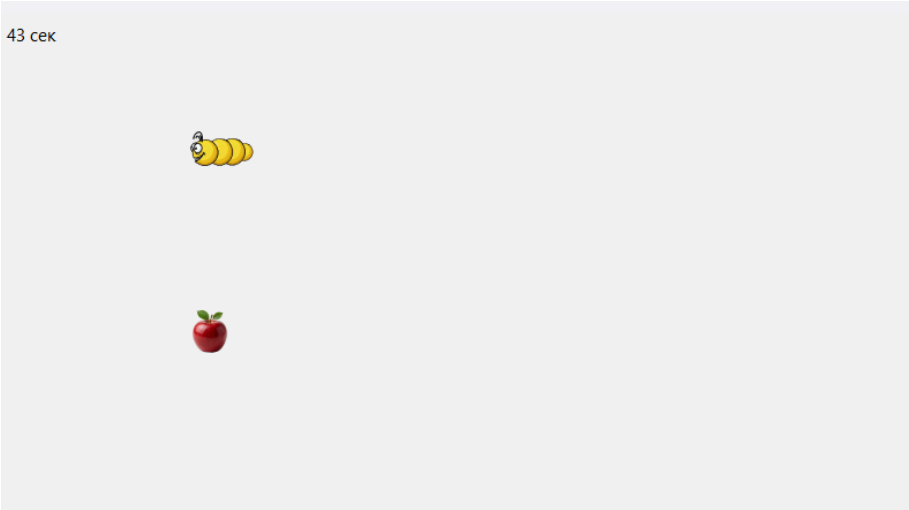


Рисунок 23 - Работа игры

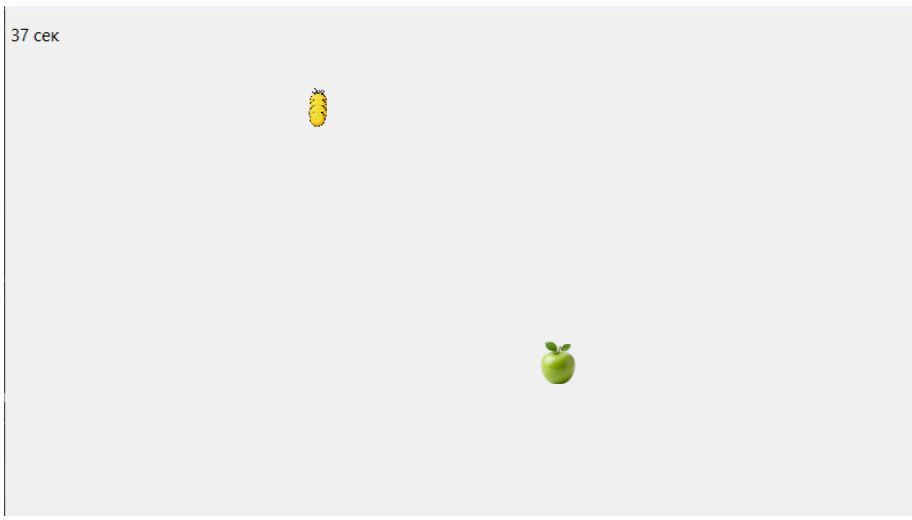


Рисунок 24 - Работа игры

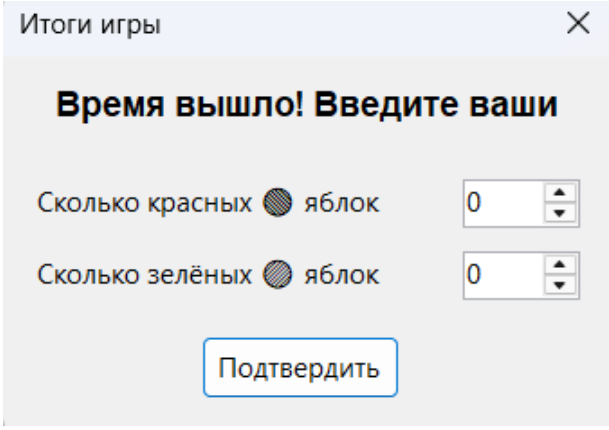


Рисунок 25 - Ввод подсчётов

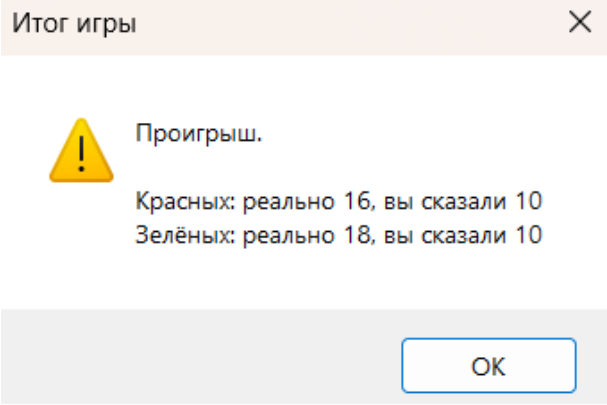


Рисунок 26 - Итоги игры